

● LIVE FEED · INTELLIGENCE TERMINAL

DOC VR-001

VERSION 1.0

\$ `init --mode=intelligence --sources=twitter,gdelt,reddit,statista,trends`

TERMINAL DE VIRALIDAD

Arquitectura, algoritmo y estrategia para detección anticipatoria de tendencias. Un sistema multi-fuente que combina Twitter (vía CloakBrowser), GDELT, Reddit, Hacker News y Google Trends con scoring de viralidad, clasificación de 4 fases y anti-gaming por capas.

/// TABLA DE CONTENIDOS

Documento de arquitectura y estrategia para la Terminal de Viralidad. Las 11 secciones que siguen fueron consensuadas tras un debate entre cinco agentes especialistas (arquitectura, algoritmo, UX/UI, fuentes, anti-gaming).

01	Resumen Ejecutivo	4
1.1	· Qué es	4
1.2	· Por qué existe	4
1.3	· Cómo funciona — pipeline en 7 etapas	4
1.4	· Diferenciadores	4
1.5	· Stack tecnológico	5
1.6	· Métricas North Star	5
02	Contexto y Visionamiento Estratégico	7
2.1	· El problema: Twitter Trending llega tarde	7
2.2	· Casos canónicos donde lead time habría generado valor	7
2.3	· Landscape competitivo	7
2.4	· Cinco ICPs (Ideal Customer Profiles)	8
2.5	· Propuesta de valor única	8
03	Síntesis del Debate Multi-Agente	9
3.1	· Los cinco agentes y sus posturas iniciales	9
3.2	· Consenso alcanzado	9
3.3	· Discrepancias resueltas	10
3.4	· Lo que NO se construye en MVP	10
04	Arquitectura del Sistema	11
4.1	· Visión general	11
4.2	· Diagrama de arquitectura	11
4.3	· Stack tecnológico justificado	12
4.4	· Pipeline de 7 etapas detallado	13

4.5 · Decisiones clave	13
4.6 · Trade-offs (puntos de tensión reales)	14
05 · Modelo de Datos y Persistencia	16
5.1 · Tabla mentions — particionada mensualmente	16
5.2 · Tabla narratives	17
5.3 · Tabla narrative_scores — serie temporal	17
5.4 · Tabla authors — hashed, no PII	18
5.5 · Tabla sources — config y salud	18
5.6 · Estrategia de particionamiento y retención	19
06 · Algoritmo de Viralidad Mejorado	20
6.1 · Crítica del algoritmo original	20
6.2 · Algoritmo mejorado — fórmula matemática formal	20
6.3 · VelocityScore — qué tan rápido crece AHORA	21
6.4 · MaturityScore — qué tan consolidada está	22
6.5 · Trash Penalty — multiplicativo de 5 sub-factores	23
6.6 · Origin Quality Score — multi-componente	23
6.7 · Decay temporal — half-life por fuente	24
6.8 · Aprendizaje de pesos — Bayesian Optimization	24
6.9 · Cross-source fusion — bayesiano con correlación	25
07 · Modelo de 4 Fases y Burst Detection	26
7.1 · Por qué no thresholds ad-hoc	26
7.2 · HMM de 4 estados	26
7.3 · Features discriminativas por fase	26
7.4 · Burst detection — CUSUM online + Kleinberg offline	27
7.5 · Resurrection probability	27
7.6 · Validación del modelo	28
08 · Fuentes y Scraping	29
8.1 · CloakBrowser 0.5.2 — qué es y cómo extenderlo	29
8.2 · Tabla de las 5 fuentes	29
8.3 · Schema unificado NormalizedMention	30

8.4 · Twitter Adapter con CloakBrowser	32
8.5 · GDELT Adapter — rate limit 1 req/5s	34
8.6 · Anti-ban strategy en 3 fases	35
8.7 · Deduplicación cross-fuente — 3 capas	36
09 Anti-Gaming y Cross-Validation	38
9.1 · Taxonomía de amenazas	38
9.2 · Penalty model en 3 capas	38
9.3 · Cross-validation matrix	39
9.4 · Reglas de legitimacy	39
9.5 · Adversarial robustness	40
10 UX/UI — Terminal de Bloomberg meets Linear	41
10.1 · Sistema de diseño — paleta completa	41
10.2 · Tipografía	42
10.3 · Layout de 5 paneles	42
10.4 · Visualización de las 4 fases	43
10.5 · Real-time UX — reglas de actualización	43
10.6 · reactbytes y typewriter — dónde usar y dónde NO	43
10.7 · Charts y visualizaciones	44
10.8 · Cuatro flujos de usuario principales	44
10.9 · Keyboard-first — shortcuts vim	45
10.10 · Anti-patrones (lo que NO haremos)	46
11 Roadmap, Métricas y Riesgos	47
11.1 · Roadmap — 3 fases	47
11.2 · Métricas North Star y KPIs por versión	47
11.3 · Tabla de riesgos y mitigaciones	48
11.4 · Ética y legalidad	48
11.5 · Modelo de negocio	49
11.6 · Cierre	49

01 Resumen Ejecutivo

La *Terminal de Viralidad* es un sistema de inteligencia en tiempo real que detecta narrativas emergentes en Twitter y las confirma cruzando GDELT, Reddit, Hacker News y Google Trends. Su propósito no es mostrar tendencias ya formadas (eso ya hace Twitter Trending), sino **adelantarse 30 minutos a 48 horas al burst público**, dando a periodistas, traders, analistas de marca e investigadores OSINT una ventana accionable.

1.1 · Qué es

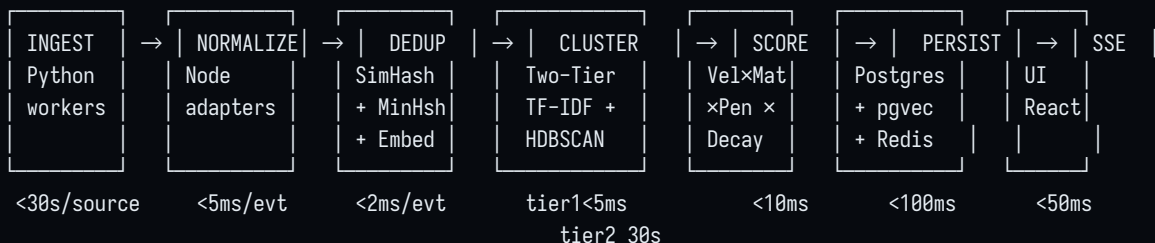
La terminal se construye como un monolito modular en Next.js 16 con workers Python desacoplados para scraping, usando CloakBrowser 0.5.2 (wrapper de Playwright con stealth a nivel binario) como única vía de acceso a Twitter sin API oficial de pago. Las cinco fuentes (Twitter, GDELT 2.0, Reddit vía OAuth2, HN Algolia, Google Trends vía pytrends) se normalizan a un schema unificado NormalizedMention, se deduplican con SimHash + MinHash + embeddings multilingües, se clusterizan en narrativas con un pipeline two-tier (TF-IDF instantáneo + HDBSCAN batch cada 30 segundos), y se puntúan con un algoritmo que desdobra velocidad y madurez.

El frontend es una terminal estilo Bloomberg con cinco paneles densos: ticker superior, lista de narrativas a la izquierda, detalle central, stream de menciones a la derecha con efecto typewriter (reactbytes), y strip de aceleración abajo. Toda la comunicación en tiempo real es vía SSE (Server-Sent Events), con throttle a 100ms vía requestAnimationFrame y cap de 50 menciones visibles. La estética es oscura profunda (#0A0E14), mono-first (SarasaMonoSC / JetBrains Mono), color 100% semántico por fase.

1.2 · Por qué existe

Twitter Trending Topics muestra lo *formado*. Para cuando una narrativa aparece ahí, ya tiene cientos de miles de menciones, la cobertura mediática la amplificó, y el momento accionable se cerró. Los casos canónicos son contundentes: **GameStop** estuvo creciendo en r/wallstreetbets durante diciembre 2020 antes del burst público de enero 2021 (lead time posible: 5-10 días). **Fyre Festival** tuvo señales en prensa local bahameña y Reddit 48 horas antes del desastre viral. **Cambridge Analytica** fue cubierta por GDELT desde diciembre de 2015, más de dos años antes del estallido en marzo 2018. Cada uno de estos casos tendría valor económico directo para un trader, un periodista, o un analista político si se hubiese detectado a tiempo.

1.3 · Cómo funciona — pipeline en 7 etapas



1.4 · Diferenciadores

Cuatro killer features distinguen este producto de cualquier clon de Twitter Trending:

- ▶ **Lead time accionable:** detección en estado "formándose" entre 30 minutos y 48 horas antes del burst público, combinando GDELT/Reddit/Google Trends como precursores débilmente acoplados.
- ▶ **Legitimacy badge transparente:** cada narrativa se etiqueta como LEGIT, BOT_CAMPAIGN, TWITTER_NATIVE o PRE_BURST mediante cross-validation bayesiana entre las 5 fuentes. Twitter no dice cuándo un trending está manipulado.
- ▶ **Cross-source attribution:** timeline que muestra dónde y cuándo apareció primero la narrativa (¿Twitter, GDELT, Reddit?), quién la originó, y cómo se amplificó.
- ▶ **Workflow curado por rol:** cinco dashboards específicos para periodista, trader cripto, analista de marca, OSINT researcher y analista político, con filtros y métricas relevantes para cada uno.

1.5
 ·
 Stack tecnológico

Capa	Tecnología	Justificación
Frontend	Next.js 16 + React 19 + reactbytes + visx	SSR, SSE nativo, animaciones terminales, charts componibles
Backend API	Next.js API Routes (Node)	Co-located con frontend, SSE gateway unificado
Scraping	Python 3.12 + CloakBrowser 0.5.2 + httpx	Stealth binario, rotación de proxies, sidecar aislado
Event bus	Redis 7 Streams + BullMQ	Consumer groups, replay, rate limiting distribuido
Persistencia	PostgreSQL 16 + pgvector 0.7	Vector search nativo, particionado mensual, JSONB
Clustering	Sidecar Python: HDBSCAN + UMAP + BGE-large	Densidad jerárquica, maneja ruido, multilingüe
Real-time	SSE (Server-Sent Events)	Unidireccional, auto-reconnect, atraviesa proxies
Monitoreo	Prometheus + Grafana	Métricas por fuente, circuit state, latencia p95

1.6
 ·
 Métricas North Star

La métrica rectora del producto es el **lead time accionable medio**: tiempo entre nuestra alerta de narrativa emergente y el burst público en Twitter Trending, ponderado por precision. Si sube, ganamos. Si baja, somos un clon caro de Twitter Trending. Los targets por versión son:

Métrica	MVP (4 sem)	v1 (3 meses)	v2 (6 meses)
Lead time accionable medio	>1h en 50% casos	>3h en 70% casos	>6h en 80% casos
Precision@k (LEGIT)	>0.80	>0.90	>0.93
Recall (BOT_CAMPAIGN)	n/a	>0.60	>0.75
False-positive bot rate	<5%	<2%	<1%
NPS por rol	—	>40	>55
Daily Active Watchlists	—	>500	>5000

NORTE ESTRATÉGICO

Sin la combinación de lead time + legitimacy badge + cross-source attribution, el producto es un clon caro de Twitter Trending. Con esos tres diferenciadores juntos, es una herramienta de intelligence con valor monetizable directo para cinco ICPs distintos.

02 Contexto y Visionamiento Estratégico

2.1 · El problema: Twitter Trending llega tarde

El producto más cercano al que aspiramos reemplazar o complementar es la pestaña "Trends" de Twitter/X. Esa lista se actualiza cada 5-10 minutos y refleja hashtags y tópicos con alto volumen *actual*. El problema operacional es que cuando un tópico llega a Trends, ya acumuló decenas de miles de menciones, fue amplificado por cuentas grandes, y muchas veces ya está siendo cubierto por medios tradicionales. El momento de oportunidad (publicar antes, invertir antes, alertar antes) se cerró.

Esto no es un defecto de implementación: es estructural. Twitter optimiza su Trending para mostrar lo que ya es socialmente significativo, no para anticipar lo que lo será. La métrica que usan es volumen bruto relativo, no velocidad de crecimiento ni calidad del originador ni cross-source amplification. Un sistema que se proponga adelantarse necesita operar con señales distintas, en ventanas temporales más cortas, y considerar fuentes externas que sirvan como precursores débilmente acoplados.

2.2 · Casos canónicos donde lead time habría generado valor

Evento	Período	Lead time posible	Señal precursora
GameStop / AMC	Dic 2020 → Ene 2021	5-10 días	r/wallstreetbets, cuentas financieras indie en Twitter
Fyre Festival	Abr 2017	48 horas	GDELT prensa local bahameña + hilos Reddit
Cambridge Analytica	Mar 2018	>2 años	GDELT desde Dic 2015, Reddit esporádico 2016-17
#MeToo	Oct 2017	6-12 horas	Activación de cuentas canario, pickup inicial
FTX collapse	Nov 2022	3-7 días	Cohort de críticos en CryptoTwitter + CoinDesk
SVB bank run	Mar 2023	12-24 horas	Twitter de VCs + Reddit r/investing

Cada uno de estos eventos comparte un patrón: existieron señales débiles en fuentes periféricas (subreddits nichos, cuentas canario, cobertura de prensa local) que precedieron al estallido masivo en Twitter. Ninguna de esas señales fue detectada en tiempo real por las herramientas comerciales existentes, porque esas herramientas miran Twitter exclusivamente y solo después de que el volumen es alto. El lead time que se perdió en cada caso corresponde a valor monetario o periodístico directo: operaciones de trading rentables, exclusivas periodísticas, mitigación de crisis de marca, inteligencia política.

2.3 · Landscape competitivo

Las herramientas existentes para monitoreo de tendencias se dividen en tres categorías, todas con limitaciones estructurales que la Terminal de Viralidad aborda:

Categoría	Ejemplos	Limitaciones
Oficiales de plataforma	Twitter Trending, Reddit Popular, HN Front	Muestran lo formado, no lo emergente; sin cross-source; sin scoring de legitimidad
Social listening SaaS	Meltwater, Brandwatch, Sprinklr, Talkwalker	Costo \$500-5000/mes, latencia horaria, APIs cerradas, no detectan pre-burst

Categoría	Ejemplos	Limitaciones
Tools gratis / OSS	TweetDeck, Trends24, RSS scraping ad-hoc	Sin algoritmo de viralidad, sin anti-gaming, sin clusterización narrativa

Ninguna combina las tres propiedades críticas: (a) fuentes múltiples con cross-validation, (b) detección temprana vía señales débilmente acopladas, (c) transparency sobre la legitimidad de cada narrativa. Ese espacio en blanco es el que ocupa la Terminal de Viralidad.

2.4 · Cinco ICPs (Ideal Customer Profiles)

El producto no es horizontal: cada audiencia tiene un workflow distinto y métricas de éxito distintas. El MVP se enfoca en uno solo (OSINT), v1 agrega los cuatro restantes.

ICP	Workflow principal	Métrica de valor
Periodista investigativo	Watchlist de tópicos + cuentas canario, busca exclusivas >24h antes que medios mainstream	# artículos con exclusiva publicada antes que competencia
Trader cripto	Filtra por cross-validation (requiere GDELT o Reddit match) + trash<0.4, opera con webhook	Lead time antes del pump >2h, Sharpe ratio de operaciones
Analista de marca	Detecta crisis tempranas con sentiment negativo + cross-pollination a comunidades nuevas	MTTR (mean time to response) reducido >50%
OSINT researcher	Cross-validation exige GDELT + Twitter/Reddit, exporta a Maltego/i2	Detección de evento geopolítico antes de medios >2h
Analista político	Detecta bot campaigns coordinadas, reporta narrativas inyectadas por redes coordinadas	% bot campaigns detectadas antes de remoción por plataforma

2.5 · Propuesta de valor única

KILLER FEATURE

No es "ver tendencias más rápido". Es **ver tendencias que todavía no están en Twitter**, cuyos precursores ya están en GDELT o Reddit, con un badge de legitimidad bayesiano que dice si la narrativa es real, manipulada, o nativa de Twitter. Eso vende a cinco audiencias con workflows distintos.

El modelo de negocio es B2B SaaS multi-ICP con tiers: free (5 alertas/día, 1 watchlist), pro (\$49-199/mes, multi-rol, alertas ilimitadas), enterprise (\$1000+/mes, white-label, API, SLA). El tier free funciona como lead magnet para capturar PMF antes de monetizar. El precio se justifica no por la cantidad de datos sino por el lead time accionable: si un trader promedio genera \$500 por operación con 2h de anticipación, cualquier precio menor a eso es obvio.

03 Síntesis del Debate Multi-Agente

Cinco agentes especialistas debatieron en paralelo sobre arquitectura, algoritmo, UX/UI, fuentes y estrategia. Cada uno defendió una postura inicial y propuso mejoras concretas. Este capítulo sintetiza el consenso final y documenta las discrepancias resueltas, para que las decisiones arquitectónicas tengan trazabilidad.

3.1 · Los cinco agentes y sus posturas iniciales

Agente	Ro1	Postura inicial defendida
#1 Arquitecto	Diseño de sistemas	Monolito modular Next.js + workers Python sidecar; Redis Streams; SSE no WebSocket; cluster two-tier con watermark 35s
#2 Científico de datos	Algoritmos / ML	Score desdoblado $Vel \times Mat$ (no mono-score); penalty multiplicativo de 5; HMM para 4 fases; CUSUM online + Kleinberg offline
#3 Designer UX/UI	Sistema de diseño	"Bloomberg Terminal meets Linear": 5 paneles densos, mono-first, color 100% semántico, keyboard-first vim
#4 Ingeniero de fuentes	Scraping / anti-ban	CloakBrowser 0.5.2 + 5 adapters con schema común; dedup 3 capas SimHash+MinHash+embeddings; circuit breakers por fuente
#5 Estratega anti-gaming	Producto / modelo de negocio	El algoritmo original es ingenuo; cross-validation bayesiana con 5 categorías de legitimacy; killer feature = lead time

3.2 · Consenso alcanzado

Tras el debate, los cinco agentes convergieron en un conjunto de decisiones que ningún agente había propuesto íntegramente al inicio. La síntesis supera a cada postura individual porque integra las preocupaciones de robustez estadística del agente #2, las restricciones operacionales del #1, los requisitos de UX del #3, las limitaciones reales de las fuentes del #4, y la visión estratégica del #5. Los puntos de consenso son:

- ▶ **Arquitectura monolito modular** con sidecars Python (no microservicios en MVP). Next.js 16 aloja frontend + API routes + SSE gateway en un proceso. Workers Python aislados para CloakBrowser y clustering.
- ▶ **Pipeline two-tier con watermark explícito de 35 segundos**: tier-1 TF-IDF asigna menciones a narrativas en menos de 5ms (provisional); tier-2 HDBSCAN cada 30 segundos reasigna y merguea con confianza.
- ▶ **Score desdoblado** $Vel(n,t) \times Mat(n,t)^{\gamma} \times Pen(n,t) \times Decay(t)$. La separación de velocidad y madurez es crítica: un solo score escalar no puede distinguir las 4 fases. γ se aprende, no se asume.
- ▶ **Penalty multiplicativo de 5 sub-factores** (bot_score, duplicate_ratio, low_quality_origin, coordinated_behavior, promotional_content). Producto, no suma: si cualquier sub-factor colapsa, el score debe ir a 0.
- ▶ **HMM de 4 estados** (forming/growing/mature/decaying) con matriz de transición estructurada (decaying absorbente). Viterbi offline para trayectorias, filter forward online para estado actual.
- ▶ **Burst detection híbrido**: CUSUM online (latencia <30s, UMbral $h \approx 5\sigma$) para alerta inmediata; Kleinberg offline batch diario para detectar jerarquías de bursts y refrescar parámetros.
- ▶ **Cross-validation bayesiana entre 5 fuentes** con 5 categorías de legitimacy: LEGIT, BOT_CAMPAIGN, TWITTER_NATIVE, PRE_BURST, NOISE. Discrepancia inter-fuente es feature explícita (no ruido).
- ▶ **SSE sobre WebSocket** para push a UI: unidireccional, auto-reconnect nativo, atraviesa proxies corporativos, 1 endpoint Next.js maneja 500-1000 conexiones sin infra extra.

- ▶ **Estética "Bloomberg Terminal meets Linear"**: dark profundo (#0A0E14), mono-first, 5 paneles densos, color semántico por fase, reactbytes reservado para momentos cognitivos (no decorativos).
- ▶ **Aprendizaje de pesos vía Bayesian Optimization** sobre precision@k@lead en walk-forward estricto. Nunca accuracy (99.99% de narrativas no son virales). Pesos por fuente, no globales.

3.3 · Discrepancias resueltas

Cinco decisiones generaron tensión real entre agentes. El siguiente cuadro documenta cada discrepancia, quién defendió cada posición, y cómo se resolvió:

Discrepancia	Posición A	Posición B	Resolución final
Twitter: API oficial vs CloakBrowser scraping	#5 Estratega: API oficial por ética	#4 Fuentes + usuario: CloakBrowser scraping	CloakBrowser + documento de compliance posture. Hashing SHA-256 con salt rotatorio para autor. Opt-out público.
Pesos del algoritmo: fijos vs aprendidos	Usuario original: pesos ad-hoc	#2 Data: Bayesian Optimization	Aprendidos via BO sobre precision@k@lead con walk-forward. Defaults razonables como cold start.
4 fases: thresholds vs HMM	Heurística simple por umbral	#2 Data: HMM de 4 estados	HMM como baseline (no requiere labels). Classifier supervisado (XGBoost) como upgrade cuando se acumulen 500-1000 trayectorias etiquetadas.
Modelo de embeddings para clustering	#4 Fuentes: paraphrase-multilingual-MiniLM (liviano)	#2 Data: BGE-large-en-v1.5 (más calidad)	BGE-large para clustering narrativo (offline, calidad). MiniLM para dedup cross-fuente (online, latencia). Dos modelos, dos propósitos.
Multi-rol en MVP vs single-ICP MVP	#5 Estratega: 5 ICPs simultáneos	#1 Arquitecto: 1 ICP (OSINT) para validar PMF	MVP single-ICP (OSINT researcher, más fácil de validar). v1 agrega los 4 restantes.

3.4 · Lo que NO se construye en MVP

Por honestidad operacional, el consenso también dejó explícito qué queda fuera del MVP para no diluir foco. Estas features se construyen en v1 o v2:

- ▶ GNN (GraphSAGE) para detección de bots coordinados. MVP usa solo heurísticas + Isolation Forest.
- ▶ Multi-idioma profundo (ES/EN/PT/FR/DE/RU/ZH). MVP cubre EN + ES nativo.
- ▶ Telegram/BlueSky/Mastodon ingestion. MVP se concentra en las 5 fuentes validadas.
- ▶ Predictive modeling (LSTM/Transformer para predecir fase futura). v2.
- ▶ Marketplace de watchlists curadas. v2.
- ▶ White-label para agencias. v2.
- ▶ Mobile app. La terminal es desktop-first por definición; mobile solo degradación elegante.

04 Arquitectura del Sistema

Arquitectura orientada a eventos con streaming en tiempo real. Monolito modular en Next.js 16 como núcleo, workers Python desacoplados para scraping y clustering, Redis Streams como bus de eventos, PostgreSQL con pgvector para persistencia. SSE para push a UI. Watermark de 35 segundos entre ingesta y narrativa visible.

4.1 · Visión general

La decisión arquitectónica central es **monolito modular, no microservicios**. Para un MVP con cinco fuentes, tráfico predecible (<1K eventos/seg) y un equipo pequeño, los microservicios introducen overhead operacional (service mesh, discovery, API gateway, observabilidad distribuida) sin aportar valor. La extracción de servicios se difiere hasta que haya presión de escala medible (>50K events/min, >70% CPU sostenido). Cuando llegue el momento, las fronteras ya están definidas: ingesters Python, cluster service, scoring service, SSE gateway, cada uno extraíble a contenedor independiente sin reescribir interfaces.

El monolito Next.js aloja en un solo proceso: el frontend React, las API routes (REST + SSE gateway), el consumidor de Redis Streams que procesa menciones normalizadas, y los servicios de scoring y persistencia. Los sidecars Python (CloakBrowser ingesters + HDBSCAN cluster) corren como procesos separados comunicados via HTTP localhost. En desarrollo se levantan con Docker Compose; en producción son dos contenedores con healthcheck mutuo. No hay service mesh, no hay API gateway, no hay discovery — es un monolito con workers.

4.2 · Diagrama de arquitectura

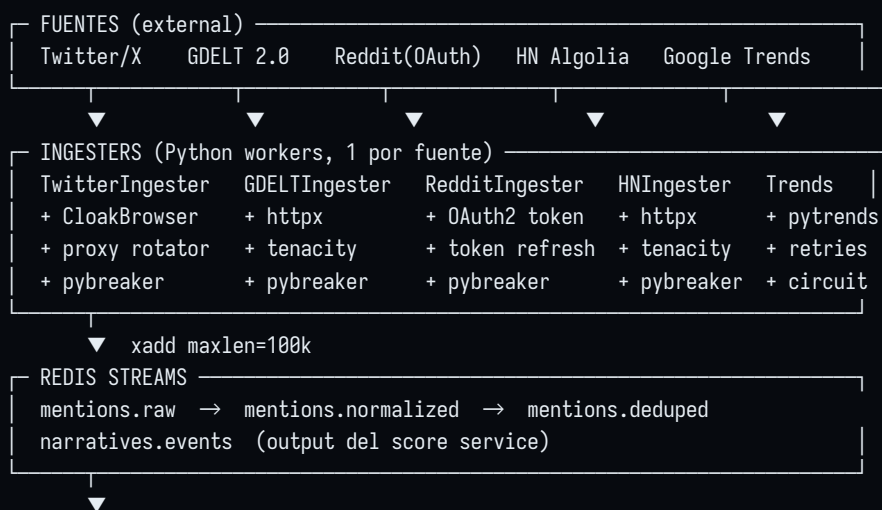
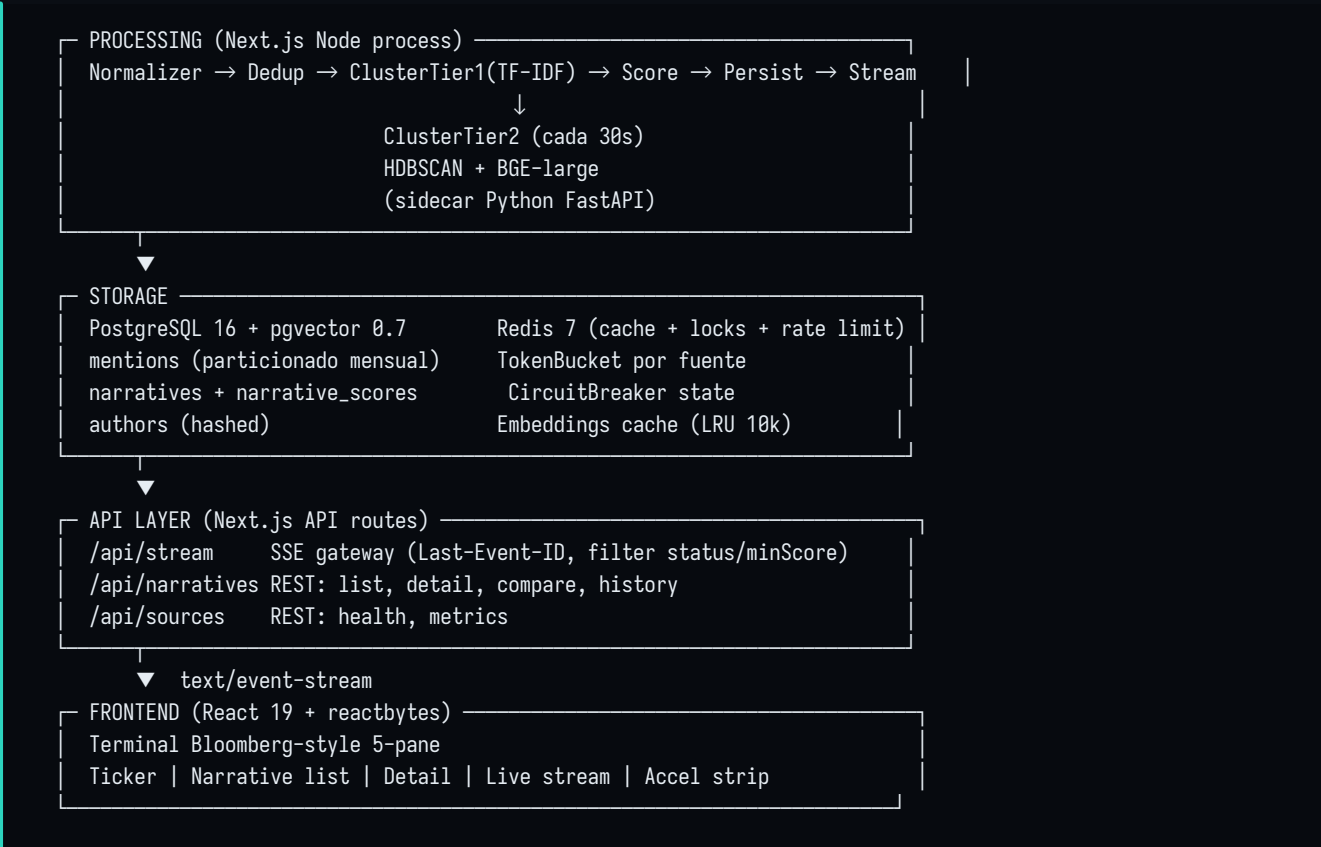


Diagrama 4.1a — Fuentes → Ingesters → Redis Streams (parte 1/2)



4.3 · Stack tecnológico justificado

Componente	Tecnología	Versión	Por qué esta elección
Frontend	Next.js	16.x	App Router, SSE en API routes, React 19 useTransition para streams
Animaciones	reactbytes + framer-motion	1.x / 11.x	Typewriter, glitch, byte-stream — efectos terminales sin CSS hacks
Charts	visx + SVG custom	0.18+	Componible, sin layout asumido, encaja en paneles densos de 320px
Backend runtime	Node.js (Bun en prod)	20.x / 1.x	SSE nativo, http2, 1 proceso maneja 500-1000 conexiones concurrentes
Scraping runtime	Python	3.12	CloakBrowser es Python-first; ecosistema scraping más maduro (httpx, tenacity, pybreaker)
Browser stealth	CloakBrowser	0.5.2+	Patches a nivel binario del Chromium, no JS injection; evita navigator.webdriver, WebGL, canvas
Bus de eventos	Redis Streams	7.x	Consumer groups + replay + persistencia. No Kafka (overkill), no RabbitMQ (routing complejo innecesario)
Rate limiting	BullMQ + Redis	5.x	Rate limiting declarativo, prioridades, retries con backoff, dashboard Bull Board
Persistencia	PostgreSQL	16.x	Particionado mensual, JSONB entities, BRIN en timestamps
Vector search	pgvector	0.7+	IVFFLAT index 384-dim resuelve 10M vectores <50ms p99. No Pinecone (vendor lock-in, latencia red)

Componente	Tecnología	Versión	Por qué esta elección
Embeddings (cluster)	BGE-large-en-v1.5	1.5	SOTA retrieval, multilingüe, 1024-dim. Solo para clustering offline
Embeddings (dedup)	paraphrase-multilingual -MiniLM-L12-v2	—	384-dim, ~120MB, CPU <15ms. Para dedup cross-fuente en hot path
Clustering	HDBSCAN + UMAP	0.8+ / 0.5+	Densidad jerárquica, robusto a ruido, no exige k. Estándar de facto para text clustering streaming
Real-time push	SSE (Server-Sent Events)	—	Unidireccional, auto-reconnect, Last-Event-ID resume, atraviesa proxies corporativos
Monitoreo	Prometheus + Grafana	2.x / 10.x	Self-hosted, gratis. Métricas por fuente, circuit state, latencia p95, queue depth

4.4 · Pipeline de 7 etapas detallado

El pipeline va desde ingesta hasta stream a UI. Cada etapa tiene un SLA explícito y el watermark total (mention ingesta → narrativa visible en UI) es p50 5s, p99 50s (cuando espera tier-2).

#	Etap	Tecnología	SLA	Output
1	INGEST	Python workers + CloakBrowser	Twitter 1req/30s/sesión×10 sesiones; otras 1req/5min	Stream mentions.raw con schema normalizado crudo
2	NORMALIZE	Node consumer, adapter pattern	<5ms/event	Stream mentions.normalized (NormalizedMention)
3	DEDUP	MinHash LSH + URL canonical	<2ms/event	Stream mentions.deduped
4	CLUSTER T1	TF-IDF cosine vs centroides	<5ms/event	mention.narrative_id provisional (tier1)
5	CLUSTER T2	Sidecar Python HDBSCAN + BGE	batch 30s, <3s interno	Reasignación final, merge de narrativas
6	SCORE	Node service on persist	<10ms/narrativa	Insert en narrative_scores + computed 4 fase
7	PERSIST+STRE AM	Postgres writer + Redis pub	p99 <100ms write, <50ms stream	Postgres row + evento a narratives.events → SSE

4.5 · Decisiones clave

Diez decisiones arquitectónicas con su alternativa descartada y justificación. Cada una fue debatida:

#	Decisión	Alternativa descartada	Justificación
1	Monolito modular Next.js 16	Microservicios en K8s / Serverless Lambda	Serverless mata SSE (cold starts, timeout 15s). K8s overhead de ops sin valor hasta 10x escala
2	Workers Python separados via Redis Streams	Todo en TypeScript (Playwright TS binding)	CloakBrowser es Python-first; aísla bans/crashes del proceso principal
3	Redis Streams como bus	Kafka / RabbitMQ / NATS	Kafka = ops pesadas. RabbitMQ = routing complejo innecesario. Streams da consumer groups + replay barato

#	Decisión	Alternativa descartada	Justificación
4	PostgreSQL 16 + pgvector	Pinecone / Weaviate / Qdrant externos	384-dim con IVFFLAT resuelve 10M vectores <50ms p99. 1 DB. Mígrar a Qdrant solo si >100M vectores
5	SSE para push a UI	WebSocket / Socket.io	Push unidireccional. SSE: auto-reconnect nativo, Last-Event-ID resume, atraviesa proxies corporativos
6	Cluster two-tier TF-IDF + HDBSCAN	Embeddings puros en cada mention	Embeddings en hot path = 50-100ms/mention = inviable a 1K mentions/s. MinHash LSH resuelve near-dup en 1-2ms
7	BGE-large para cluster, MiniLM para dedup	Un solo modelo para todo	BGE-large (1024-dim) pesado pero calidad para cluster offline. MiniLM (384-dim) liviano para dedup online
8	pgvector con IVFFLAT + BRIN en timestamps + GIN en JSONB	HNSW siempre / B-tree everywhere	HNSW más rápido pero rebuild caro y memoria-intensive. IVFFLAT suficiente para 10M. BRIN 10x más chico que B-tree para timestamps append-only
9	BullMQ + pybreaker	Celery / RQ puro / sin cola	BullMQ da rate limiting declarativo, prioridades, retries con backoff, dashboards. pybreaker es el circuit breaker estándar Python
10	Tablas particionadas mensualmente + TTL Redis 7d	Tabla única creciendo / sin TTL	mentions crece 1-5M filas/mes. DROP PARTITION para retención 90d sin VACUUM pesado. Eventos crudos en Redis con TTL 7d

4.6 · Trade-offs (puntos de tensión reales)

Tres trade-offs que el equipo aceptó conscientemente, con la estrategia de mitigación para cada uno:

T1 · FRESCURA vs COSTO DE SCRAPING

*Detectar "narrativas formándose" exige alta frecuencia (poll cada 30s en Twitter). Pero más frecuencia = más proxies, más sesiones, más costo, más riesgo de ban. **Mitigación:** scraping adaptativo con feedback loop — narrativas activas en estado forming/rising con score>40 suben a poll 15s; narrativas formed/declining bajan a 5min. Reduce costo ~60% vs polling uniforme agresivo manteniendo frescura donde importa.*

T2 · LATENCIA DE CLUSTERING vs CALIDAD SEMÁNTICA

*HDBSCAN + embeddings detecta mejor narrativas emergentes que TF-IDF, pero tarda 2-5s por batch. Si espero el batch, pierdo frescura; si uso TF-IDF solo, fallo en cross-fuente. **Mitigación:** two-tier con watermark explícito de 35s. Tier-1 asigna mención a narrativa en <5ms y la UI la muestra como "forming" con flag cluster_confidence=provisional. Tier-2 reasigna y mergea cada 30s. El usuario acepta 35s de ruido a cambio de no perder 35s de detección.*

T3 • SIMPLICIDAD DEL MONOLITO vs AISLAR PYTHON

*Quiero un solo proceso (monolito Next.js), pero CloakBrowser y HDBSCAN viven mejor en Python. Levantar 2 runtimes rompe el "monolito". **Mitigación:** sidecars Python con FastAPI en localhost HTTP, no microservicios. En Docker Compose es 1 servicio con sidecar. En prod son 2 contenedores con healthcheck mutuo. No hay service mesh, no hay API gateway, no hay discovery. Es un monolito con workers, no una red de microservicios.*

05 Modelo de Datos y Persistencia

Cinco tablas principales: *mentions* (particionada mensualmente), *narratives*, *narrative_scores* (serie temporal), *authors* (hashed), *sources* (config y salud). Índices críticos: *IVFFLAT* en *embeddings*, *BRIN* en *timestamps*, *GIN* en *JSONB entities*. Retención 90 días vía *DROP PARTITION*.

5.1 · Tabla *mentions* — particionada mensualmente

La tabla central del sistema. Cada mención individual (tweet, artículo de GDELT, post de Reddit, story de HN, tendencia de Google Trends) es una fila. Se particiona mensualmente por *fetched_at* para permitir *DROP PARTITION* eficiente (retención 90 días) sin *VACUUM* pesado. Estimación: 1-5M filas/mes.

SQL

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE mentions (
  id          BIGSERIAL,
  source      TEXT NOT NULL,          -- 'twitter' | 'gdel't' | 'reddit' | 'hackernews' | 'googletrends'
  external_id TEXT NOT NULL,          -- id nativo de la fuente
  url         TEXT,
  url_canonical TEXT,                  -- para dedup cross-fuente
  author_hash TEXT NOT NULL,          -- SHA-256(salt_rotatorio + author_id) — no PII
  author_handle TEXT,                  -- solo display, no persistente >30d
  content     TEXT NOT NULL,
  language    TEXT,                  -- ISO 639-1
  entities     JSONB,                  -- {persons:[], orgs:[], locations:[]}
  embedding    vector(384),            -- all-MiniLM-L6-v2 (dedup) — BGE-large en narrative centroid
  minhash_sig  BYTEA,                 -- datasketch MinHash 128 perms
  narrative_id BIGINT,                 -- FK narratives, NULL hasta clusterizar
  cluster_state TEXT DEFAULT 'unclustered', -- 'unclustered' | 'tier1' | 'final' | 'noise'
  engagement   JSONB,                 -- {likes, retweets, comments, ...} fuente-specific
  fetched_at   TIMESTAMPTZ NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (id, fetched_at)
) PARTITION BY RANGE (fetched_at);

-- Partición auto-creada por script monthly
CREATE TABLE mentions_2026_07 PARTITION OF mentions
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');

-- Índices críticos
CREATE INDEX idx_mentions_url_canon  ON mentions (url_canonical) WHERE url_canonical IS NOT NULL;
CREATE INDEX idx_mentions_narrative  ON mentions (narrative_id) WHERE narrative_id IS NOT NULL;
CREATE INDEX idx_mentions_created_brin ON mentions USING BRIN (created_at);
CREATE INDEX idx_mentions_entities_gin ON mentions USING GIN (entities jsonb_path_ops);
CREATE INDEX idx_mentions_source_ext  ON mentions (source, external_id);

-- IVFFLAT para 384-dim: lists = sqrt(rows) ~ 1000 para 1M vectores
CREATE INDEX idx_mentions_embedding ON mentions
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 1000);
```

5.2 · Tabla narratives

Cada narrativa es un cluster de menciones semánticamente cohesivas. Mantiene métricas rolling pre-calculadas (velocity_1h/6h/24h, acceleration, entropy) para scoring rápido sin recomputar desde mentions cada vez. El status captura la fase actual (forming/rising/formed/declining/dead) según el HMM.

SQL

```
CREATE TABLE narratives (
  id          BIGSERIAL PRIMARY KEY,
  title       TEXT,                -- auto-generada: top entities + verbo dominante
  summary     TEXT,                -- LLM-generated opcional
  status      TEXT NOT NULL DEFAULT 'forming',
                                -- 'forming' | 'rising' | 'formed' | 'declining' | 'dead'
  origin_source TEXT,              -- primera fuente que la detectó
  origin_quality REAL DEFAULT 0.5, -- 0-1, calidad del iniciador
  legitimacy   TEXT DEFAULT 'UNCERTAIN', -- 'LEGIT' | 'BOT_CAMPAIGN' | 'TWITTER_NATIVE' | 'PRE_BURST' | 'NOISE'
  first_seen   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  mention_count INTEGER NOT NULL DEFAULT 0,
  author_count INTEGER NOT NULL DEFAULT 0,
  source_count INTEGER NOT NULL DEFAULT 0, -- # de fuentes distintas que la confirmaron
  -- métricas rolling pre-calculadas
  velocity_1h  REAL DEFAULT 0,      -- menciones/hora última 1h
  velocity_6h  REAL DEFAULT 0,
  velocity_24h REAL DEFAULT 0,
  acceleration REAL DEFAULT 0,      -- segunda derivada del volumen
  entropy      REAL DEFAULT 0,      -- H(authors) Shannon normalizada
  trash_penalty REAL DEFAULT 1.0,    -- 0-1 (1 = limpio, 0.3 = spam)
  current_score REAL DEFAULT 0,      -- Vel × Mat^γ × Pen × Decay
  burst_onset  TIMESTAMPTZ,          -- momento CUSUM disparó (t*)
  predicted_peak TIMESTAMPTZ,        -- predicción de peak (cuando HMM transicione a formed)
  centroid_embedding vector(1024),   -- BGE-large, se actualiza incrementalmente
  keywords     TEXT[],               -- top 10 TF-IDF
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_narratives_status_score ON narratives (status, current_score DESC);
CREATE INDEX idx_narratives_last_seen_brin ON narratives USING BRIN (last_seen);
CREATE INDEX idx_narratives_keywords_gin ON narratives USING GIN (keywords);
CREATE INDEX idx_narratives_centroid ON narratives
  USING ivfflat (centroid_embedding vector_cosine_ops) WITH (lists = 500);
```

5.3 · Tabla narrative_scores — serie temporal

Historial temporal del score y sus componentes por narrativa. Permite reconstruir la trayectoria que llevó a la fase actual, esencial para backtesting, validación del HMM, y visualización en el timeline del detalle.

SQL

```
CREATE TABLE narrative_scores (
  narrative_id BIGINT NOT NULL REFERENCES narratives(id),
  score REAL NOT NULL,          -- score final compuesto
  velocity_score REAL NOT NULL, -- Vel(n,t) en [0,1]
  maturity_score REAL NOT NULL, -- Mat(n,t) en [0,1]
  trash_penalty REAL NOT NULL,  -- Pen(n,t) en [0,1]
  decay_factor REAL NOT NULL,   -- Decay(t-t0) en [0,1]
  phase TEXT NOT NULL,          -- 'forming' | 'rising' | 'formed' | 'decaying'
  phase_confidence REAL DEFAULT 0.5, -- confianza del HMM en la clasificación
  features JSONB,               -- vector de features completo para backtesting
  computed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (narrative_id, computed_at)
) PARTITION BY RANGE (computed_at);

-- TimescaleDB hypertable cuando se habilite la extensión:
-- SELECT create_hypertable('narrative_scores', 'computed_at');
```

5.4 · Tabla authors — hashed, no PII

Para compliance con GDPR, los autores se hashcan con SHA-256 y salt rotatorio mensual (no reversible, no es PII bajo Art. 4). El handle display se persiste solo 30 días en Redis (no en Postgres). La reputation se computa agregado, no se guarda per-author PII. El bot_score combina heurísticas + Isolation Forest + features de grafo (GNN en v1).

SQL

```
CREATE TABLE authors (
  author_hash TEXT PRIMARY KEY,      -- SHA-256, no reversible
  first_seen TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  mention_count INTEGER DEFAULT 0,
  source_counts JSONB,               -- {twitter: 120, reddit: 3}
  reputation REAL DEFAULT 0.5,       -- 0-1, recalculado por batch diario
  is_bot_suspect BOOLEAN DEFAULT FALSE,
  bot_score REAL DEFAULT 0,          -- 0-1, heurísticas + Isolation Forest
  origin_quality REAL DEFAULT 0.5,   -- 0-1, quality como iniciador de narrativas
  account_age_days INTEGER,          -- estimado, no PII
  follower_following_ratio REAL      -- feature anti-bot
);
CREATE INDEX idx_authors_bot ON authors (is_bot_suspect) WHERE is_bot_suspect;
CREATE INDEX idx_authors_reputation ON authors (reputation DESC);
```

5.5 · Tabla sources — config y salud

Configuración y estado de cada fuente. Permite al panel admin mostrar salud en tiempo real (latencia, success rate, last fetch, circuit state) y al scheduler decidir cadencias adaptativas.

SQL

```
CREATE TABLE sources (
  name          TEXT PRIMARY KEY,      -- 'twitter', 'gdelt', etc.
  enabled       BOOLEAN DEFAULT TRUE,
  fetch_interval_s INTEGER NOT NULL,   -- 30, 300, etc.
  last_fetch_at TIMESTAMPTZ,
  last_success_at TIMESTAMPTZ,
  consecutive_errors INTEGER DEFAULT 0,
  circuit_state TEXT DEFAULT 'closed', -- 'closed' | 'open' | 'half_open'
  circuit_opened_at TIMESTAMPTZ,
  success_rate_1h REAL DEFAULT 1.0,
  latency_p95_ms REAL DEFAULT 0,
  config        JSONB                  -- {proxy_pool, session_count, ...}
);
```

5.6 · Estrategia de particionamiento y retención

Dos tablas crecen sin bound: mentions (1-5M filas/mes) y narrative_scores (~10K filas/día). Ambas se particionan mensualmente. La retención es 90 días para mentions y 1 año para narrative_scores. La partición del mes corriente tiene autovacuum agresivo (scale_factor=0.05) porque los inserts son append-only y continuos. Las particiones viejas se dropean sin VACUUM:

SQL

```
-- Cron job mensual: crea partición del mes siguiente, dropea la de hace 4 meses
CREATE OR REPLACE FUNCTION manage_partitions() RETURNS void AS $$
DECLARE
  next_month_start DATE := date_trunc('month', NOW() + INTERVAL '1 month');
  next_month_end   DATE := date_trunc('month', NOW() + INTERVAL '2 months');
  old_partition    TEXT;
BEGIN
  EXECUTE format(
    'CREATE TABLE IF NOT EXISTS mentions_%s PARTITION OF mentions FOR VALUES FROM (%L) TO (%L)',
    to_char(next_month_start, 'YYYY-MM'), next_month_start, next_month_end);
  SELECT tablename INTO old_partition FROM pg_tables
  WHERE tablename LIKE 'mentions_%' AND tablename < to_char(NOW() - INTERVAL '4 months', 'mentions_YYYY-MM')
  ORDER BY tablename LIMIT 1;
  IF old_partition IS NOT NULL THEN
    EXECUTE format('DROP TABLE IF EXISTS %s', old_partition);
  END IF;
END;
$$ LANGUAGE plpgsql;
```

06 Algoritmo de Viralidad Mejorado

El algoritmo propuesto por el usuario ($S = 100 \cdot (w1 \cdot z(v) + w2 \cdot norm(a) + w3 \cdot H + w4 \cdot origin_quality) \cdot penalty$) es una base correcta pero insuficiente. Esta sección presenta la crítica detallada, el algoritmo mejorado con fórmulas matemáticas formales, y la justificación de cada componente.

6.1 · Crítica del algoritmo original

Componente	Crítica	Mejora propuesta
z(v) burst velocidad	Z-score asume Gaussianidad y ventana fija. Frágil ante volatilidad diaria y seasonal patterns.	Robust z-score con mediana + MAD; ventana adaptativa con STL decomposition; combinar con CUSUM online
norm(a) aceleración	Segunda derivada del volumen es ruidosísima en series discretas pequeñas. Sensible a 1-2 spikes.	Suavizado Savitzky-Golay o filtro Kalman antes de derivar. Normalizar contra aceleración mediana histórica
H(authors) entropía	Colineal con volumen: H crece con N. No distingue 1000 autores orgánicos de 1000 bots coordinados.	Entropía normalizada $H/\log(N)$ + complementar con network_velocity (PageRank narrativo) + coordinated_behavior_score
origin_quality	Solo mide el primer poster. En Twitter el originador real suele ser ambiguo (RTs, quote tweets, screenshots).	PageRank sobre el grafo de cascada completo + eigenvector centrality del seed + tiempo al primer retweet de alto PageRank
penalty(trash)	Único factor escalar aditivo. No descompone modos de ataque distintos (bots vs duplicates vs coordinación).	Producto multiplicativo de 5 sub-penales: bot_score $\times$ dup_ratio $\times$ low_origin $\times$ coordination $\times$ promotional
Pesos w1..w4 fijos	No calibrados. El usuario los está adivinando. Diferentes fuentes tienen dinámicas distintas.	Aprender vía regresión logística L2 + isotonic calibration, o Bayesian Optimization sobre precision@k@lead
Ausencia de Decay	Narrativa viral de hace 72h sigue con score alto. Terminal muestra narrativas muertas.	$Decay(t) = \exp(-\lambda(t-t_0))$, half-life $\tau_{1/2} = \ln(2)/\lambda$ con λ por fuente (Twitter 6-12h, GDELT 48h)
Ausencia de cross-source	Narrativa que solo existe en Twitter pero no en GDELT/Reddit es sospechosa o weak signal.	Bayesian fusion con likelihoods por fuente + correlación inter-fuente como feature
Ausencia de semantic novelty	Tema ya visto 50 veces, aunque tenga volumen, no es emergente.	SemanticNovelty = $1 - \max_sim(emb, centroids_históricos)$ con KNN sobre embeddings históricos
Mono-score escalar	Un único score no puede distinguir las 4 fases (alto score puede ser creciendo o formada).	Separar en VelocityScore y MaturityScore; clasificar fase con HMM de 4 estados
Sin ground truth	Imposible saber si el algoritmo funciona.	Backtesting sobre eventos etiquetados (GDELT GKG, MemeTracker, Hoaxy/Snopes)

Lo robusto que vale conservar: (a) el esqueleto multiplicativo score $\times$ penalty es correcto, permite que una sola categoría de trash anule la señal; (b) el uso de entropía de Shannon para dispersión es estándar y sólido; (c) el z-score de velocidad es razonable si se robustifica.

6.2 · Algoritmo mejorado — fórmula matemática formal

Sea n una narrativa, t el tiempo discreto (ventanas Δt de 5-15min para Twitter, 1h para GDELT), $s \in \{tw, gd, rd, hn, gt\}$ una fuente. El score final compuesto es:

FORMULA

$$S(n,t) = 100 \cdot Vel(n,t) \cdot Mat(n,t)^\gamma \cdot Pen(n,t) \cdot Decay(t - t_{sub}^{0} \cdot t_{super}^n)$$

donde:

$Vel(n,t) \in [0,1]$ — velocidad de crecimiento actual (momentum)
 $Mat(n,t) \in [0,1]$ — consolidación acumulada (maturity)
 $\gamma \in [0.3, 0.7]$ — peso de madurez vs momentum (APRENDIDO)
 $Pen(n,t) \in [0,1]$ — producto de 5 sub-penales anti-trash
 $Decay \in [0,1]$ — $\exp(-\lambda \cdot \Delta t)$, half-life por fuente

6.3 · VelocityScore — qué tan rápido crece AHORA

El VelocityScore captura el momentum actual de la narrativa. Es una sigmoide sobre una combinación lineal de 6 features:

FORMULA

$$Vel(n,t) = \sigma(w_{sub1} \cdot \tilde{z}(v) + w_{sub2} \cdot \tilde{acc} + w_{sub3} \cdot \hat{H}(A) + w_{sub4} \cdot NetVel + w_{sub5} \cdot SemNov + w_{sub6} \cdot XSrc(n,t) + b)$$

$\tilde{z}(v)$ — **robust z-score del volumen** usando mediana + MAD en vez de media + desvío:

FORMULA

$$\tilde{z}(v) = \frac{v_{\{n,t\}} - \text{median}_{\{\tau \in W\}}(v_{\{n,\tau\}})}{1.4826 \cdot \text{MAD}_{\{\tau \in W\}}(v_{\{n,\tau\}}) + \epsilon}$$

W es ventana histórica (24-72h). El factor 1.4826 hace MAD estimador consistente del desvío Gaussian. Esto es robusto a outliers y a picos puntuales que no son bursts reales.

$\tilde{acc}$ — **aceleración normalizada** sobre volumen suavizado:

FORMULA

$$\tilde{acc} = \frac{\hat{v}_{\{n,t\}} - 2 \cdot \hat{v}_{\{n,t-1\}} + \hat{v}_{\{n,t-2\}}}{\sigma_{acc}(n) + \epsilon}$$

$\hat{v}$ es volumen suavizado con filtro Kalman o Savitzky-Golay. $\sigma_{acc}(n)$ es la desviación histórica de la aceleración de n . Sin suavizado, la segunda derivada de series discretas pequeñas es ruidosísima.

$\hat{H}(A)$ — **entropía de Shannon normalizada** de la distribución de autores:

FORMULA

$$\hat{H}(A) = \frac{- \sum_{a \in A} p_a \cdot \log(p_a)}{\log(|A|)}$$

La normalización por $\log(|A|)$ corrige el sesgo pro-volumen: 1000 autores únicos orgánicos y 1000 bots coordinados tendrían H similar sin normalizar, pero con $\hat{H}$ se distinguen mejor. $p_a = \text{count}_a / \sum \text{count}$.

NetVel — **network velocity (PageRank narrativo)** midiendo cómo crece la centralidad del seed:

FORMULA

$$\text{NetVel}(n, t) = d/dt [\text{PR}_{\{G_{\{n, t\}}\}}(u_{\star})]$$

$G_{\{n, t\}}$ es el subgrafo de cascada de la narrativa (nodos=usuarios, aristas=RT/quote/reply), $u_{\star}$ el seed (originador atribuido). Mide cómo crece la centralidad del seed — captura "este tweet está siendo amplificado por nodos influyentes", no solo por volumen.

SemNov — **semantic novelty** midiendo qué tan inesperado es el tema:

FORMULA

$$\text{SemNov}(n, t) = 1 - \max_{\{c \in C_{\text{hist}}\}} \cos(e_n, \mu_c)$$

C_{hist} es el set de centroides de narrativas históricas (últimos 30 días), e_n el embedding promedio de n . 1 = totalmente inesperado, 0 = tópico ya conocido. Esto permite priorizar narrativas nuevas sobre repeticiones de temas ya vistos.

XSrc — **cross-source amplification** bayesiano:

FORMULA

$$\text{XSrc}(n, t) = P(\text{viral} \mid e^{(tw)}, e^{(gd)}, e^{(rd)}, e^{(hn)}, e^{(gt)}) - P_{\text{sub} > 0 \text{sub} <}$$

Posterior bayesiano centrado respecto al prior P_0 . Si la narrativa fue detectada en múltiples fuentes, XSrc sube. Si solo en Twitter, baja (potencial bot campaign o nativa de Twitter).

6.4 · MaturityScore — qué tan consolidada está

MaturityScore captura cuán consolidada está la narrativa. Es ortogonal a Vel: una narrativa puede tener Vel alto pero Mat bajo (formándose), o Vel bajo pero Mat alto (formada/decaída).

FORMULA

$$\text{Mat}(n, t) = \sigma(\alpha_{\text{sub} > 1 \text{sub} <} \cdot \log(\sum v_{\{n, t\}}) + \alpha_{\text{sub} > 2 \text{sub} <} \cdot \hat{H}(A) + \alpha_{\text{sub} > 3 \text{sub} <} \cdot |S_{\text{conf}}(n, t)| + \alpha_{\text{sub} > 4 \text{sub} <} \cdot \text{depth}(G_{\{n, t\}}) + \alpha_{\text{sub} > 5 \text{sub} <} \cdot \text{age}(n, t)_{\text{capped}})$$

Término	Significado	Rango
$\Sigma v_{\{n,\tau\}}$	Volumen acumulado (consolidación por repetición)	$\log(0)$ a $\log(\infty)$
$\hat{H}(A)$	Entropía media integrada en el tiempo (diversidad sostenida)	0 a 1
$ S_{\text{conf}}(n,t) $	Número de fuentes distintas que confirmaron n	0 a 5
$\text{depth}(G_{\{n,t\}})$	Profundidad media del árbol de cascada (cascadas profundas = consolidadas)	0 a ∞
$\text{age}(n,t)_{\text{capped}}$	Edad saturada (no premiar puramente la antigüedad)	0 a T_{max}

6.5 · Trash Penalty — multiplicativo de 5 sub-factores

El penalty es el componente más importante del algoritmo. Sin él, el sistema se vuelve un espejo de bots. Es un **producto** (no suma) porque si cualquier sub-factor colapsa a 0, el score total debe ir a 0:

FORMULA

$$\text{Pen}(n,t) = \prod_{i=1..5} p_i(n,t), \quad p_i \in [0,1]$$

Sub-factor p_i	Significado	Modelo / fuente
p_1 bot_score	Fracción estimada de cuentas bot	Isolation Forest + graph features (degree, reciprocity, account age) + clasificador Botometer-like
p_2 duplicate_ratio	Ratio de menciones near-duplicate	MinHash / SimHash sobre texto normalizado, Jaccard threshold 0.85
p_3 low_quality_origin	1 - autoridad del seed	Invertir origin_quality normalizado (ver §6.6)
p_4 coordinated_behavior	Score de coordinación temporal	Cross-correlation de timestamps por pares de usuarios + clustering espectral
p_5 promotional_content	Ratio de contenido comercial/spam	URL ratio, hashtag stuffing, classifier (regex + transformer)

PRODUCTO, NO SUMA

El producto (no la suma) es crítico: si cualquier sub-factor colapsa a 0, el score total debe ir a 0. Con suma, un solo término bajo no aniquila el score. Esto significa que una narrativa con bot_score=0.1 pero alta calidad en otros aspectos NO se promociona al top del feed, porque el producto la penaliza completamente.

6.6 · Origin Quality Score — multi-componente

El origin_quality no debe ser solo "PageRank del primero". Se descompone en 6 componentes que capturan distintas dimensiones de calidad del iniciador:

FORMULA


```

origin_quality(initiator) = 0.25·history_in_topic
                           + 0.20·engagement_quality
                           + 0.20·network_position
                           + 0.15·account_authenticity
                           + 0.10·temporal_priority
                           + 0.10·cross_source_corroboration
    
```

Componente	Cálculo	Peso
history_in_topic	% posts históricos del iniciador sobre el tema (últimos 90d)	0.25
engagement_quality	Ratio (engagement/followers) normalizado, peso replies > likes	0.20
network_position	PageRank + betweenness en grafo del tema	0.20
account_authenticity	1 – bot_probability(GNN), edad, perfil, historial 90d	0.15
temporal_priority	1 si verdadero primer post público (no recycled)	0.10
cross_source_corroboration	Aparece en GDELT/Reddit con timestamp cercano	0.10

Caso ilustrativo: hashtag político #X. Escenario A — periodista con 5 años en el tema: history_in_topic=0.9, engagement_quality=0.8, network_position=0.85, account_authenticity=0.95 → origin_quality = 0.85. Escenario B — cuenta nueva, 12k followers, 0 engagement: history_in_topic=0.1, engagement_quality=0.05, account_authenticity=0.2 → origin_quality = 0.10. La narrativa se relega hasta corroboración. Los bots macedonios de 2016 tenían cuentas creadas 3-6 meses antes → origin_quality ≈ 0.18 → correcta penalización.

6.7 · Decay temporal — half-life por fuente

Sin decay, narrativas viejas siguen con score alto y la terminal se llena de muertos. Decay exponencial suave y diferenciable:

FORMULA

```

Decay(Δt) = exp(-λ · Δt),    λ = ln(2) / τ½
    
```

Fuente	Half-life τ½ sugerido	Justificación
Twitter	6-12 h	Vida media de trending topics
Reddit	18-24 h	Frontpage rotation
Hacker News	24 h	Ciclo daily
GDELT	48 h	Cadencia de cobertura mediática
Google Trends	72 h	Reporting lag

Cuando el HMM detecta estado "decaída", se acelera el decay multiplicando λ por β_decay ∈ [2,5]. Así una narrativa en fase de muerte desaparece rápido del ranking. Exponential decay es preferible a sliding window puro porque es suave y diferenciable, ideal para gradient-based learning de λ.

6.8 · Aprendizaje de pesos — Bayesian Optimization

Los pesos w₁..w₆ y α₁..α₅ NO son fijos. Se aprenden vía dos métodos complementarios:

- **Regresión logística L2 + isotonic calibration:** target binario $y_{\{n,t\}} = 1$ si n alcanza "viralidad real" ($v_{\{n,t\}} \geq \theta_{\text{viral}}$ dentro de ventana $\Delta T_{\text{horizon}} = 6h$). Esto define lead time supervisado.
- **Bayesian Optimization sobre precision@k@lead:** cuando la métrica de negocio no es diferenciable. BO con GP surrogate + Expected Improvement, 50-200 evaluaciones, Optuna. Útil porque precision@k@lead es no diferenciable y ruidosa.

Anti-overfitting crítico: walk-forward estricto (entrenar en $[T_0, T_1]$, validar en $[T_1, T_1 + \Delta]$, deslizar; nunca aleatorizar), L2 fuerte ($\sim 10^{-3}$) por colinealidad entre features, Bayesian priors $N(0, \sigma^2)$ para estabilizar.

6.9 · Cross-source fusion — bayesiano con correlación

Sea $e^{\{s\}}_t \in \{0,1\}$ la evidencia "fuente s muestra actividad anómala de n en t ". Bajo asunción de independencia condicional (falsa pero útil como baseline):

FORMULA

$$P(\text{viral} \mid e^{\{tw\}}, \dots, e^{\{gt\}}) = P(\text{viral}) \cdot \prod_s P(e^{\{s\}} \mid \text{viral}) / P(e^{\{tw\}}, \dots, e^{\{gt\}})$$

Posterior odds (forma cómoda para logs):

FORMULA

$$\begin{aligned} \log [P(\text{viral} \mid e) / P(\sim \text{viral} \mid e)] = & \log [P(\text{viral}) / P(\sim \text{viral})] \\ & + \sum_s \log [\text{TPR}^{\{s\}} / \text{FPR}^{\{s\}}] \cdot e^{\{s\}} \\ & + \sum_s \log [(1 - \text{TPR}^{\{s\}}) / (1 - \text{FPR}^{\{s\}})] \cdot (1 - e^{\{s\}}) \end{aligned}$$

La asunción de independencia es falsa (Twitter y Reddit se influyen mutuamente). Solución: stacking con LightGBM como meta-learner sobre predicciones per-source. El meta-modelo aprende implícitamente las correlaciones. Además, la varianza inter-fuente se usa como feature explícita:

FORMULA

$$\text{Discrepancy}(n,t) = \text{Var}_s [v_{e1}^{\{s\}}(n,t)]$$

Alta discrepancia = señal de manipulación monocanal o de evento source-specific. Esto es un feature, no ruido: cuando Twitter muestra alta velocidad pero GDELT no, algo raro está pasando.

07 Modelo de 4 Fases y Burst Detection

Las 4 fases (formándose / creciente / formada / decaída) se modelan como estados latentes de un HMM, no como umbrales ad-hoc del score. El burst detection usa CUSUM online (latencia <30s) + Kleinberg offline (jerarquía de bursts, batch diario). El lead time se define como $t_{alert} - t_{peak}$.

7.1 · Por qué no thresholds ad-hoc

Cualquier regla tipo " $Vel > 0.7 \Rightarrow$ creciente" es frágil: los rangos cambian por fuente, por temporada, por tipo de evento. Necesitamos un modelo que capture la dinámica temporal como transición entre estados latentes. El HMM (Hidden Markov Model) es el modelo canónico para esto: estados ocultos (las 4 fases) que emiten observaciones (el vector de features), con matriz de transición que captura la progresión natural forming $\rightarrow$ rising $\rightarrow$ formed $\rightarrow$ decaying.

7.2 · HMM de 4 estados

Estados latentes $S = \{s_{form}, s_{grow}, s_{mat}, s_{dec}\}$. Matriz de transición estructurada (las narrativas progresan, no saltan arbitrariamente):

FORMULA

$$A = \begin{bmatrix} 0.85 & 0.13 & 0 & 0.02 \\ 0 & 0.80 & 0.18 & 0.02 \\ 0 & 0 & 0.85 & 0.15 \\ 0 & 0 & 0 & 1.00 \end{bmatrix} \begin{array}{l} \leftarrow \text{forming} \\ \leftarrow \text{growing} \\ \leftarrow \text{mature} \\ \leftarrow \text{decaying (ABSORBENTE)} \end{array}$$

a_fd pequeño (algunas narrativas mueren sin madurar)
 a_dd = 1 (estado absorbente — narrativa decaída no revive)
 Si revive, se considera nueva instancia de narrativa

Emisiones: Gaussianas multivariadas sobre el vector de features observables $o_t = (Vel, Mat, dv/dt, \Delta H, XSrc) \in \mathbb{R}^5$. $b_j(o_t) = N(o_t; \mu_j, \Sigma_j)$. Inferencia: Viterbi para estimar la trayectoria más probable de estados (offline, para backtesting); filter forward para estimación online del estado actual.

7.3 · Features discriminativas por fase

Fase	Vel	Mat	dv/dt	ΔH	XSrc
Formándose	medio-alto	bajo	> 0 , creciente	creciente	bajo-medio (1 fuente)
Creciente	alto	medio	> 0 , pico	alto, estable	medio (2-3 fuentes)
Formada	medio-bajo	alto	≈ 0	alto, estable	alto (≥ 3 fuentes)
Decaída	bajo	alto (decay)	< 0	decay	decay

Esta tabla muestra cómo las features discriminan las fases. La combinación (Vel alto + Mat bajo + $dv/dt > 0$ + XSrc bajo) es la firma inequívoca de "formándose" — la narrativa está creciendo rápido pero aún no está consolidada ni confirmada por múltiples fuentes. Es exactamente el momento donde el producto agrega valor.

7.4 · Burst detection — CUSUM online + Kleinberg offline

El burst detection identifica el momento exacto en que una narrativa "explota". Se usan dos algoritmos complementarios:

Algoritmo	Tipo	Online?	Captura	Cuándo usar
CUSUM	Sequential test	Sí	Punto de cambio (cambio de media)	Online low-latency en la terminal (<30s)
Kleinberg	HMM gamma-states	Offline	Jerarquía de bursts multi-escala	Backtesting, batch diario, refrescar parámetros
PELT	Changepoint penalizado	Offline	Múltiples changepoints	Segmentación post-hoc para análisis
Adams-Mackay	Bayesian online	Sí	Run-length posterior	Cuando hay incertidumbre cuantificable

CUSUM one-sided robust con baseline EWMA (no asume media constante):

FORMULA

```

S_t<sup>+</sup> = max(0, S_{t-1}<sup>+</sup> + (v_{n,t} - μ<sub>0</sub>)/σ - k)
Alarma si S_t<sup>+</sup> > h

k (slack)      ≈ δ/2 · σ, donde δ es magnitud mínima de shift a detectar (0.5σ-2σ)
h (umbral)     se calibra vía ARL (Average Run Length), típicamente h ∈ [4σ, 6σ]
μ<sub>0</sub>, σ      se estiman online vía EWMA: μ<sub>0</sub> ← μ<sub>0</sub> + α·(v - μ<sub>0</sub>)
```

Kleinberg modela los gaps entre posts como exponenciales con tasa α_i en estado i. Un burst es transición a estado de mayor α. Detecta jerarquías de bursts (sub-bursts dentro de bursts), útil para distinguir "creciendo" de "pico". Cuando CUSUM dispara alarma en t*, marcamos t* como burst onset. El origin_quality se recalcula mirando el autor del primer post antes de t* (no el primer post de la narrativa, porque a veces la narrativa duerme días antes de explotar). El lead time del sistema:

FORMULA

```

Δt_lead = t_alert - t_peak
```

Negativo = anticipación correcta. Se reporta la distribución (no solo la media) porque los outliers positivos (detección muy temprana) son los casos de mayor valor.

7.5 · Resurrection probability

Una narrativa en estado "decaída" puede revivir (resurrection). El HMM estándar con estado absorbente no permite esto, así que se modela como una nueva instancia. Pero computacionalmente queremos detectar el revival: si en los últimos 7 días una narrativa decaída muestra σ re-emerging > 0.3, se computa:

FORMULA

```

P(resurrection | decaying, σ_re-emerging > 0.3) ≈ P(rising | features)
```

Si la probabilidad es >20%, la UI muestra un banner sutil "WATCH THIS — resurrection 12%". Esto es útil para el flujo "explorar narrativa decaída" del usuario que busca segundas olas.

7.6 · Validación del modelo

Métricas para validar el HMM y el burst detector:

Métrica	Definición	Cuándo importa
Precision@k@lead	Fracción de Top-k alertas que fueron virales reales Y con anticipación $\geq \delta_{\text{min}}$ al peak	Métrica principal de negocio
Recall@k	Fracción de eventos virales reales capturados en Top-k	Si el costo de perderse un evento es alto
Lead time	$\Delta t_{\text{lead}} = t_{\text{alert}} - t_{\text{peak}}$ (negativo = anticipación correcta)	Distribución, no solo media
False Positive Rate	# alertas no-virales / # no-virales totales	Costo operacional de falsos positivos
MCC	Matthews Correlation Coefficient para fases (balanceado ante clases desbalanceadas)	Evaluación del HMM
Brier score	Calibración de probabilidades	Si se usan scores como probabilidades
ARL	Average Run Length para CUSUM (latencia esperada de detección)	Tuning del burst detector

Backtesting metodológico: walk-forward estricto (prohibido aleatorizar timestamps), sin look-ahead bias (cualquier feature que use info posterior a t debe ser eliminado — cuidado con z-score usando mediana que incluye el futuro), event-level evaluation (no post-level), stratified by source, ablation studies (correr el score sin SemNov, sin XSrc, sin Decay para medir contribución marginal).

08 Fuentes y Scraping

Cinco fuentes asimétricas con un patrón SourceAdapter común. CloakBrowser 0.5.2 (wrapper de Playwright con stealth a nivel binario) es la base exclusiva para Twitter. GDELT 2.0 impone 1 req/5s verificado. Reddit requiere OAuth2 obligatorio (el .json sin auth está degradado). HN Algolia funciona perfecto sin key. Google Trends necesita pytrends. Deduplicación cross-fuente en 3 capas.

8.1 · CloakBrowser 0.5.2 — qué es y cómo extenderlo

CloakBrowser es un package npm mantenido por cloakhq, licencia MIT. Latest publicada: 0.5.3 (se recomienda actualizar desde 0.5.2). Peer-deps: playwright-core ^1.40 y puppeteer-core ^21. Tamaño unpacked ~37KB (wrapper liviano; el binario Chromium es de Playwright). Descripción oficial: "Stealth Chromium that passes every bot detection test. Drop-in Playwright/Puppeteer replacement with source-level fingerprint patches."

A diferencia de puppeteer-extra-plugin-stealth (que inyecta JS para evadir detección), CloakBrowser modifica el binario compilado de Chromium. Esto evita navigator.webdriver, leaks de chrome.runtime, permissions.query, WebGL vendor/renderers, canvas noise, AudioContext fingerprint, font enumeration, hardware concurrency, device memory. Es drop-in replacement: misma API que Playwright (browser.newPage(), page.goto()).

Limitaciones verificadas: (1) no incluye proxies — hay que pasarlos vía --proxy-server; (2) no rota user agents automáticamente; (3) no resuelve CAPTCHAs (integrar 2captcha/CapMonster externo); (4) no tiene modo "human behavior" built-in — scroll, hover, mouse movement hay que programarlos encima; (5) binario pesado ~200MB por versión; (6) no evade login wall de Twitter; (7) update lag — dependés del maintainer para releasear patches cuando Twitter cambia detección.

8.2 · Tabla de las 5 fuentes

Fuente	Endpoint	Frecuencia	Latencia	Rate limit	Reliability
Twitter/X	x.com/search?q=...&f=live (HTML)	30s-1min por query	3-8s	Variable, soft-ban risk	Media
GDELT DOC 2.0	api.gdeltproject.org/api/v2/doc	15 min	0.5-2s	1 req/5s por IP	Alta
Reddit OAuth2	oauth.reddit.com/r/{sub}/hot	5 min por sub	0.3-1s	60 req/min con OAuth	Alta
HN Algolia	hn.algolia.com/api/v1/search	10 min	0.2-0.8s	Sin límite documentado	Muy alta
Google Trends	trends.google.com/trends/api/...	1 hora	2-5s	~1 req/10s recomendado	Baja (429 frecuente)

NOTAS CRÍTICAS VERIFICADAS EN RUNTIME

Reddit .json sin auth ahora devuelve HTML aunque el UA sea correcto formato "linux:app:ver (by /u/user)". OAuth2 obligatorio desde v1. GDELT devuelve texto plano "Please limit requests to one every 5 seconds..." si se viola el rate. Google Trends raw curl devuelve 404 — solo funciona con pytrends que replica headers/tokens internos.

8.3 • Schema unificado NormalizedMention

Cada fuente implementa un adapter que normaliza a un schema común. El campo raw es el escape hatch que evita migrar el schema cada vez que una fuente añade campos:

TYPESCRIPT

```

export type SourceType = 'twitter' | 'gdelt' | 'reddit' | 'hackernews' | 'googletrends';
export type MentionType = 'post' | 'article' | 'story' | 'comment' | 'trend_signal';

export interface NormalizedMention {
  // Identidad
  id: string; // UUID interno
  source: SourceType;
  source_id: string; // ID nativo (tweet_id, objectID, reddit fullname)
  url: string; // URL canónica
  fetched_at: number; // epoch ms
  published_at: number | null; // epoch ms (cuándo se publicó)

  // Contenido
  type: MentionType;
  title: string | null; // null para tweets
  body: string; // texto principal
  lang: string | null; // ISO 639-1
  entities: {
    hashtags?: string[];
    cashtags?: string[];
    mentions?: string[];
    urls?: string[];
    domains?: string[]; // gdelt
  };
  media?: { type: 'image' | 'video' | 'none'; urls: string[] }[];

  // Autor
  author: {
    id: string | null;
    handle: string | null;
    name: string | null;
    followers?: number; // solo twitter
    karma?: number; // reddit/hn
  };

  // Engagement (no comparable cross-fuente)
  engagement: {
    likes?: number; retweets?: number; replies?: number;
    score?: number; // reddit/hn upvotes
    comments?: number;
    views?: number; // twitter impressions
  };

  // Geo (GDELT lo da, twitter rara vez)
  geo?: { country?: string; state?: string; city?: string; lat?: number; lon?: number };

  // GDELT themes
  themes?: string[]; // 'ENV_CLIMATE', 'TAX_FNCACT', ...
  tone?: number; // GDELT tone -100..+100

  // Dedup
  content_hash?: string; // SimHash 64-bit hex
  embedding?: number[]; // vector 384-d (MiniLM) lazy

  // Escape hatch
  raw: Record<string, unknown>; // payload original sin normalizar
}

```


8.4 · Twitter Adapter con CloakBrowser

El adapter de Twitter es el más complejo por el login wall y el anti-bot. Estrategia: cookies exportadas manualmente de 3-5 cuentas, rotación round-robin, humanización encima de CloakBrowser (scroll + hover + pausas randomizadas). Paginación incremental con since_id (el tweet_id más alto) agregado al query.

TYPESCRIPT · TWITTERADAPTER (PARTE 1/2 — FETCH PRINCIPAL)

```
import { connect, type Browser } from 'cloakbrowser';
import type { NormalizedMention, SourceAdapter } from '../types';

export class TwitterAdapter implements SourceAdapter {
  private browser: Browser | null = null;

  constructor(private cookies: any[], private proxyPool: ProxyPool) {}

  async fetch(params: { query: string; maxTweets?: number; sinceId?: string }): Promise<NormalizedMention[]> {
    if (!this.browser) {
      this.browser = await connect({
        headless: true,
        proxy: this.proxyPool.next(),
        args: ['--disable-blink-features=AutomationControlled'],
      });
    }
    const ctx = await this.browser.newContext({
      userAgent: uaPool.next(),
      viewport: { width: 1920, height: 1080 },
      locale: 'en-US',
      timezoneId: 'America/New_York',
    });
    await ctx.addCookies(this.cookies);
    const page = await ctx.newPage();

    const query = params.sinceId ? `${params.query} since:${params.sinceId}` : params.query;
    await page.goto(`https://x.com/search?q=${encodeURIComponent(query)}&f=live`,
      { waitUntil: 'networkidle', timeout: 30000 });
    await this.humanize(page); // scroll + hover + pausas
    await page.waitForSelector('article[data-testid="tweet"]', { timeout: 15000 });

    // Scroll incremental hasta coleccionar maxTweets
    const collected: NormalizedMention[] = [];
    let lastHeight = 0;
    while (collected.length < (params.maxTweets ?? 50)) {
      const tweets = await this.extractTweets(page);
      for (const t of tweets) {
        if (!collected.find(c => c.source_id === t.source_id)) {
          collected.push(t);
          if (collected.length >= (params.maxTweets ?? 50)) break;
        }
      }
      await page.evaluate(() => window.scrollBy(0, 1500));
      await sleep(1500 + Math.random() * 2000);
      const h = await page.evaluate(() => document.body.scrollHeight);
      if (h === lastHeight) break;
      lastHeight = h;
    }
    await ctx.close();
    return collected;
  }
}
```

TYPESCRIPT · TWITTERADAPTER (PARTE 2/2 — HUMANIZE + EXTRACTTWEETS)

```

// Humanización: scroll + hover + pausas randomizadas sobre CloakBrowser
private async humanize(page: Page) {
  const rand = (min: number, max: number) => min + Math.random() * (max - min);
  await page.mouse.move(rand(100, 1500), rand(100, 800));
  await sleep(rand(200, 600));
  for (let i = 0; i < rand(3, 7); i++) {
    await page.mouse.wheel(0, rand(200, 800));
    await sleep(rand(700, 2000));
    if (Math.random() < 0.2) {
      const els = await page.$$('a', article, [role="button"]);
      if (els.length) await els[Math.floor(Math.random() * els.length)].hover();
      await sleep(rand(300, 900));
    }
  }
}

// Extracción de tweets desde el DOM via $$eval
private async extractTweets(page): Promise<NormalizedMention[]> {
  return await page.$$eval('article[data-testid="tweet"]', (arts) => {
    return arts.map((a) => {
      const link = a.querySelector('a[href*="/status/"]') as HTMLAnchorElement;
      const href = link?.href ?? '';
      const idMatch = href.match(/status\/(\d+)/);
      const text = a.querySelector('[data-testid="tweetText"]')?.textContent ?? '';
      const handle = a.querySelector('a[role="link"] span')?.textContent ?? '';
      const nums = (sel: string) => parseInt(
        a.querySelector(sel)?.getAttribute('aria-label')?.replace(/\D/g, '') ?? '0', 10);
      return {
        id: crypto.randomUUID(), source: 'twitter' as const,
        source_id: idMatch?.[1] ?? '', url: href,
        fetched_at: Date.now(), published_at: null,
        type: 'post' as const, title: null, body: text, lang: null,
        entities: {},
        author: { handle: handle.replace('@', ''), name: null, id: null },
        engagement: {
          replies: nums('[data-testid="reply"]'),
          retweets: nums('[data-testid="retweet"]'),
          likes: nums('[data-testid="like"]'),
        },
        raw: { html: a.outerHTML.slice(0, 5000) },
      } as NormalizedMention;
    });
  });
}

```

8.5 · GDELT Adapter — rate limit 1 req/5s

GDELT DOC 2.0 API devuelve artículos. Modo artlist para mentions, mode=timelinevol para volumen temporal. Sintaxis de query: ("climate change" OR "global warming") sourcecountry:US. Operadores: AND, OR, NOT, sourcecountry:, sourceclang:, domain:.

TYPESCRIPT

```
const GDELT_URL = 'https://api.gdeltproject.org/api/v2/doc/doc';

export class GdeltAdapter implements SourceAdapter {
  constructor(private rl: TokenBucket) {} // rate limit 1 req / 5s

  async fetch(params: { query: string; maxrecords?: number }): Promise<NormalizedMention[]> {
    await this.rl.take('gdelt', 5000); // VERIFICADO: 1 req/5s por IP
    const u = new URL(GDELT_URL);
    u.searchParams.set('query', params.query);
    u.searchParams.set('mode', 'artlist');
    u.searchParams.set('format', 'json');
    u.searchParams.set('maxrecords', String(params.maxrecords ?? 250));
    u.searchParams.set('sort', 'datedesc');

    const res = await fetch(u.toString(), {
      headers: { 'User-Agent': 'TerminalDeViralidad/0.1 (research)' },
    });
    if (!res.ok) throw new Error(`GDELT ${res.status}`);
    const data = await res.json();

    return (data.articles ?? []).map((a: any): NormalizedMention => ({
      id: crypto.randomUUID(),
      source: 'gdelt',
      source_id: a.url, // GDELT no tiene ID propio
      url: a.url,
      fetched_at: Date.now(),
      published_at: parseGdeltDate(a.seendate),
      type: 'article',
      title: a.title,
      body: a.title, // GDELT DOC no da body
      lang: a.language ?? null,
      entities: { domains: [a.domain], urls: [a.url] },
      author: { handle: null, name: a.domain, id: null },
      geo: a.country ? { country: a.country } : undefined,
      raw: a,
    })));
  }
}
```

8.6 · Anti-ban strategy en 3 fases

Capa	Gratis	Pago	Cuándo pagar
User-Agent rotation	✓ Libs: user-agents, fake-useragent	—	Nunca
Browser fingerprint rotation	✓ CloakBrowser ya lo hace	—	Nunca
Human behavior (scroll/hover)	✓ Custom impl	—	Nunca
Request delays randomizados	✓ Custom + jitter	—	Nunca
Proxies datacenter	✓ Listas gratis (frágiles)	\$50-100/mes BrightData	Casi nunca — datacenter no sirve para Twitter

Capa	Gratis	Pago	Cuándo pagar
Proxies residenciales	✗ No hay gratis buenos	\$100-500/mes	Cuando X/Reddit bloquean >5% de requests o escala >10K req/día
Proxies mobile (4G/5G)	✗	\$300-1000/mes	Solo si Twitter soft-banea sistemáticamente
CAPTCHA solver	2captcha ~\$1.5/1000	hCaptcha/Turnstile más caro	Cuando login flow ocasional requiere CAPTCHA
Cookie/session farm	✓ Manual con 3-5 cuentas	Agencias \$20/cuenta	Cuando 1 sola cuenta se satura

Fase 1 (MVP, gratis): CloakBrowser + UA rotation + 3 cookies exportadas manualmente + delays 3-8s random entre scrolls + sin proxies. **Objetivo:** ~500 tweets/día sin ban. **Fase 2 (producción baja):** añadir proxies residenciales (BrightData pay-as-you-go ~\$10/GB ≈ 10K requests) + cookie farm 10-20 cuentas + circuit breaker que desactive cuenta ante 3 errores 429/403. **Objetivo:** 10K-50K tweets/día. **Fase 3 (escala):** proxies mobile 4G rotating + cookie farm 50+ cuentas con perfiles distintos + solver externo. **Objetivo:** 100K+ tweets/día.

8.7 · Deduplicación cross-fuente — 3 capas

Una misma noticia aparece en Twitter, Reddit, HN, GDELT con URLs distintas. Hay que dedup por contenido semántico, no por URL. Tres capas: SimHash (cheap blocking) → MinHash (confirmación Jaccard) → embeddings multilingües (cross-idioma y cross-dominio). Las capas son jerárquicas: solo se corre la capa N si la N-1 produjo candidatos.

TYPESCRIPT

```
// Capa 1: SimHash 64-bit (cheap blocking, O(1) con LSH bucketing)
// Umbral: Hamming distance ≤ 3 bits → candidato
function simhash64(text: string): bigint {
  const tokens = tokenize(text); // lowercase, sin URLs/mentions/hashtags
  const v = new Int32Array(64); // weighted sum per bit
  for (const tok of tokens) {
    const h = fnv1a64(tok);
    for (let i = 0; i < 64; i++) {
      if (((h >> BigInt(i)) & 1n) === 1n) v[i]++;
      else v[i]--;
    }
  }
  let out = 0n;
  for (let i = 0; i < 64; i++) if (v[i] > 0) out |= (1n << BigInt(i));
  return out;
}

// Capa 2: MinHash + LSH (Jaccard similarity ≥ 0.6)
// Sobre shingles de 3 caracteres o tokens. 128 perms, ~2KB por mention.

// Capa 3: Embeddings multilingües (cross-idioma, cross-dominio)
// Modelo: paraphrase-multilingual-MiniLM-L12-v2 (384-d, ~120MB, CPU <15ms)
// Cosine similarity ≥ 0.78 → mismo tema aunque tweet en español y GDELT en inglés
// Solo se corre para candidatos que pasaron capa 1 o 2 (lazy).
```

Parámetro	Valor recomendado	Justificación
SimHash Hamming threshold	3	99.6% sim, ~0.04% false positives
Embedding cosine threshold	0.78 multilingüe	0.85 si todo es inglés
MinHash Jaccard	0.6	Balance precision/recall para near-dup
Longitud mínima de texto	40 caracteres	Evitar matches sobre "RT @user:"

09 Anti-Gaming y Cross-Validation

El *penalty(trash)* es el componente más importante del algoritmo. Sin él, el sistema replica manipulación de bots. Tres capas: heurísticas deterministas + Isolation Forest + GNN GraphSAGE (v1). Cross-validation bayesiana entre 5 fuentes con 5 categorías de legitimacy.

9.1 · Taxonomía de amenazas

Amenaza	Caso real	Detección
Bot campaign coordinado	Macedonios en 2016 US election	Isolation Forest + graph features
Astroturfing	Russia IRA, #WalkAway	GNN GraphSAGE sobre grafo de interacción
Manipulación pagada	Fyre Festival promo, influencers pagos	origin_quality bajo + sentiment uniformity
Spam promocional	Crypto shill accounts	URL ratio + hashtag stuffing classifier
Recycled content	Fake news elections	MinHash duplicate_ratio + media hash
Hashtag hijacking	#MeToo hijack attempts	Sentiment shift + comunidad outlier
Coordinated inauthentic behavior	Twitter integrity reports	Cross-correlation temporal + clustering espectral

9.2 · Penalty model en 3 capas

El *penalty* es un stack de tres capas que se combinan en un producto. Cada capa captura distintos modos de ataque, así evadir una no evada las otras.

Capa	Método	Latencia	Re-entrenamiento	Precisión esperada
1 — Heurísticas	8 reglas deterministas sobre 21 features	<5ms	Manual, on-demand	Alta (alta precision, baja recall)
2 — Isolation Forest	Anomalía no supervisada sobre 21 features	<50ms	Semanal sobre narrativas legítimas	Media (captura outliers)
3 — GNN GraphSAGE	Sobre grafo de interacción (cuentas+RT)	<200ms	Quincenal con labels de Twitter integrity	Alta (captura coordinación)

Capa 1 — Heurísticas deterministas (rápidas, alta precision):

PYTHON

```

def heuristic_trash_score(features):
    flags = []
    if features['new_account_ratio'] > 0.4: flags.append(('new_accounts', 0.3))
    if features['synchronized_posting_ratio'] > 0.2: flags.append(('sync_posting', 0.4))
    if features['duplicate_content_ratio'] > 0.3: flags.append(('dup_content', 0.4))
    if features['network_density_among_posters'] > 0.7 \
        and features['louvain_cluster_concentration'] > 0.85:
        flags.append(('coordinated_cluster', 0.5))
    if features['sentiment_uniformity'] < 0.05 \
        and features['hashtag_jaccard_mean'] > 0.7:
        flags.append(('copy_paste_campaign', 0.4))
    if features['hour_of_day_uniformity'] < 0.1: flags.append(('non_human_cadence', 0.3))
    if features['url_shortener_ratio'] > 0.5: flags.append(('spammy_urls', 0.2))
    if features['media_hash_duplicate_ratio'] > 0.4: flags.append(('media_recycle', 0.3))
    score = 1 - prod(1 - w for _, w in flags) # 0=limpio, 1=basura
    return min(score, 1.0), flags
    
```

Capa 2 — Isolation Forest: anomalía no supervisada, ~50ms inference. Entrenado por semana sobre narrativas etiquetadas como legítimas. Score >0.7 → trash. Captura outliers que las heurísticas no codifican explícitamente. Requiere re-entrenamiento semanal porque los bots evolucionan.

Capa 3 — GNN GraphSAGE (v1): sobre grafo de interacción. Nodos = accounts, edges = menciones/retweets en ventana 6h. Etiquetas: cuentas confirmadas como bots por Twitter integrity reports (dataset público). Agregación a nivel narrativa. Re-entrenamiento quincenal. La combinación final:

FORMULA

```

trash(n) = 0.4·heuristic + 0.3·isolation_forest + 0.3·gnn_aggregate
    
```

trash score	Acción
< 0.3	Narrativa legítima, sin penalty
0.3 - 0.6	Flag de atención (badge "anómalo" en UI)
≥ 0.6	Ocultar del feed principal, cola de revisión

9.3 · Cross-validation matrix

Cada fuente tiene una latencia distinta respecto a Twitter y un nivel de confianza base distinto. La cross-validation bayesiana combina estas señales:

Fuente	Latencia vs Twitter	Señal detectable	Confianza base	Peso Bayesian
Twitter API	0 (referencia)	Volumen, autores, sentimiento	0.5 (contaminable)	0.3
GDELT 2.0	1-24h antes	Prensa local/global, eventos geopol	0.85	0.3
Reddit (nichos)	2-48h antes	Discusión profunda, filtraciones	0.75	0.2
Hacker News	6-48h antes	Tech, startups, seguridad	0.8	0.1
Google Trends	1-12h antes	Interés de búsqueda general	0.6	0.1

9.4 · Reglas de legitimacy

La combinación de fuentes detectadas + trash score produce 5 categorías de legitimacy. Estas categorías son la killer feature del producto: Twitter no te dice cuándo un trending está manipulado.

PYTHON

```
def legitimacy(narrative):
    s = sources_detected    # {twitter, gdelt, reddit, hn, g_trends}
    trash = compute_trash(narrative)

    if s.twitter and (s.gdelt or s.redddit) and trash < 0.4:
        return 'LEGIT', 0.92    # Confirmado multi-fuente, alta confianza
    if s.twitter and not (s.gdelt or s.redddit) and trash > 0.5:
        return 'BOT_CAMPAIGN', 0.85    # Solo Twitter + alto trash = bots
    if s.twitter and not (s.gdelt or s.redddit) and trash < 0.4:
        return 'TWITTER_NATIVE', 0.55    # Solo Twitter pero limpio = rumor/meme
    if (s.gdelt or s.redddit) and not s.twitter:
        return 'PRE_BURST', 0.70    # GDEL/Reddit sin Twitter = PRE-BURST
        # ★ esto es lo que queremos detectar ★
    if not s.twitter and not s.gdelt and not s.redddit:
        return 'NOISE', 0.2
    return 'UNCERTAIN', 0.5
```

CASO REAL — CAMBRIDGE ANALYTICA

GDELT pickup desde diciembre 2015. Reddit esporádico 2016-2017. Twitter burst sólo en marzo 2018 tras Channel 4. Con cross-validation, el sistema habría detectado esto como PRE_BURST desde diciembre 2015 — lead time posible >2 años. Ninguna herramienta comercial detectó esto en tiempo real.

9.5 • Adversarial robustness

Los bots evolucionan para evadir las capas del penalty. Mitigaciones:

- ▶ **Randomizar ligeramente los umbrales τ por hora** — no dar información exacta a un atacante sobre qué lo dispara.
- ▶ **Auditoría periódica** — re-entrenar p_1 (bot_score) y p_4 (coordinated_behavior) cada 7 días para seguir el drift de tactics bot.
- ▶ **Ensemble diverso** — combinar Isolation Forest + Botometer + heurísticas. Evadir una capa no evada las otras.
- ▶ **PSI (Population Stability Index)** por feature, alerta si >0.25. KL divergence entre distribución de scores histórica vs rolling 24h. Re-entrenar cuando PSI >0.25 en ≥ 2 features.

10 UX/UI — Terminal de Bloomberg meets Linear

Estética Bloomberg Terminal (densidad, multi-panel, mono-first) cruzada con Linear (cuidado tipográfico, microinteracciones sutiles, keyboard-first). Fondo #0A0E14, color 100% semántico por fase, reactbytes reservado para momentos cognitivos. Keyboard-first con shortcuts vim (j/k/enter/g/s/c/f/r/m/?).

10.1 · Sistema de diseño — paleta completa

Veinticuatro tokens de color con hex exactos. Backgrounds, borders, texto, acentos por fase, funcionales. Cada color tiene un rol semántico, no estético. Jamás usar #2DD4BF para un estado que no sea rising.

Token	Hex		Uso
bg.base	#0A0E14	Fondo raíz	
bg.elevated	#0D1117	Paneles nivel 1	
bg.panel	#11161D	Paneles nivel 2	
bg.hover	#161B22	Hover de fila/celda	
bg.active	#1C2128	Selección, table header	
bg.inset	#070A0F	Inputs, code blocks	
border.subtle	#1F2937	Bordes pasivos	
border.default	#21262D	Separadores entre paneles	
border.strong	#30363D	Bordes de panel activo	
border.focus	#5EEAD4	Focus ring (con alpha 40%)	
text.primary	#E6EDF3	Títulos, números clave (15.4:1 AAA)	
text.secondary	#94A3B8	Labels, metadata (5.1:1 AA)	
text.tertiary	#7D8590	Hints, placeholders	
text.disabled	#484F58	Disabled	
Fase / Función	Token	Hex primario	Glow (con alpha)
Formándose	amber	#FBBF24	#FBBF2433
Creciente	teal	#2DD4BF	#5EEAD440
Formada	slate	#94A3B8	—
Decaída	rose	#F87171	#F8717130
Live	live	#00FF9F	#00FF9F66
Link	link	#58A6FF	—

Fase / Función	Token	Hex primario	Glow (con alpha)
Warning	warn	#F59E0B	—
Danger	danger	#EF4444	—

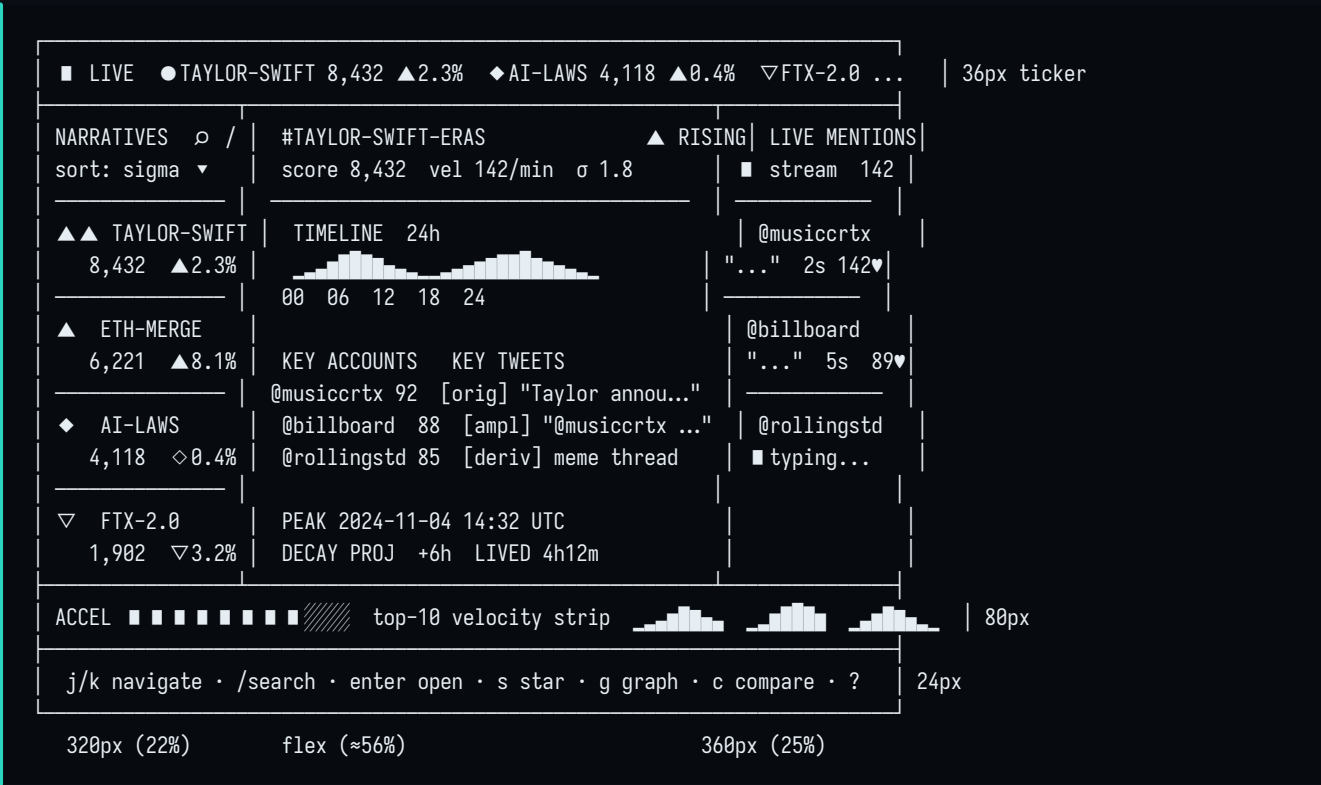
10.2 · Tipografía

Dos familias: mono para datos, sans para prosa. Nunca mezcladas en la misma jerarquía. Mono para scores, timestamps, IDs, ticker, code. Sans para descripciones, captions, copy de UI.

Familia	Uso	Escala
Mono (JetBrains Mono / IBM Plex Mono fallback)	Datos, scores, IDs, timestamps, ticker, code	xs 10 / sm 11 / base 12 / md 13 / lg 15 / xl 18 / 2xl 24 px
Sans (Inter / Geist Sans fallback)	Descripciones, captions, copy UI, prosa	xs 11 / sm 12 / base 13 / md 14 / lg 16 / xl 20 px

Letter-spacing: mono en -0.01em para datos tabulares (alineá mejor); sans en 0 default, -0.011em para títulos grandes. Spacing grid de 4px. Padding interno de paneles: 12px (densidad Bloomberg). Radius pequeño deliberado (default 4px) — refuerza el feel terminal, evita parecer SaaS genérico.

10.3 · Layout de 5 paneles



Wireframe 10.1 — Terminal default con 5 paneles (1440×900)

Jerarquía visual para evitar abrumar: (1) fila activa tiene borde teal 2px + glow sutil — es el foco atencional; (2) ticker tiene opacidad 0.85 por defecto, full opacity solo al hover — periférico; (3) stream derecho usa tipografía 11px y opacidad 0.9 → baja prioridad atencional; (4) accel strip decorativo-periférico con opacidad 0.7; (5) hint bar opacidad 0.5 — casi invisible hasta que el usuario lo necesita.

10.4 · Visualización de las 4 fases

Fase	Color	Icono	Animación	Badge
Formándose	#FBBF24	◇	Pulse lento 2.4s ease-in-out; glow tenue borde izq	◇ FORMING σ0.9
Creciente	#2DD4BF	▲	Score glitch 1 frame al cambio >5%; byte-stream en burst	▲ RISING +1.8σ
Formada	#94A3B8	●	Sin animación (quieto); marca temporal de pico fija	● PEAKED 14:32
Decaída	#F87171	▽	Opacidad 1→0.85; borde dashed 1px; strike-through sutil	▽ DECAYING -3.2%/h

Regla semántica de color: jamás usar #2DD4BF para un estado que no sea rising. Jamás usar #F87171 para algo que no sea decay. Los funcionales (error, warning, info) usan hex distintos (#EF4444, #F59E0B, #38BDF8) para no contaminar el significado de fase. Transiciones de fase animadas con framer-motion cross-fade 240ms.

10.5 · Real-time UX — reglas de actualización

Evento	Cadencia	Acción UI
Nueva mención individual	push SSE	Inserta al top del right panel con typewriter 60cps; si panel lleno (>50 items), elimina último con fade-out 180ms
Cambio de score ≤5%	push SSE	Count-up animado del número (200ms ease-out), tinte efímero teal/rose
Cambio de score >5%	push SSE	Count-up + glitch 1 frame (reactbytes) en el número
Burst event (>20% / CUSUM)	push SSE	Byte-stream particle burst 800ms sobre la fila + sonido blip (si activo)
Cambio de fase	push SSE	Cross-fade del PhaseBadge 240ms + borde izquierdo recolorea
Nuevo narrative (top-K)	push SSE	Inserta al top de left panel con slide-down 12px + fade-in
Ticker refresh	throttle 1s	Reordena y actualiza deltas

Throttle y debounce: right panel menciones batch en ventanas de 100ms vía requestAnimationFrame (si llegan 30 menciones en 100ms, se insertan como bloque con stagger 16ms entre cada una — evita el "tsunami" visual). Left panel re-sort throttle 1s con reordenamiento animado via framer-motion layout (spring stiffness 300, damping 30). Ticker throttle 1s. Charts re-render throttle 500ms. Pausa inteligente con visibilitychange: si document.hidden, stream.pause() + buffer.accumulate(); al volver, pinta los últimos 50 items sin animación.

10.6 · reactbytes y typewriter — dónde usar y dónde NO

Momento	Lib/reactbytes API	Intensidad	Duración
Boot sequence		alta	3-5s
Nueva mención en stream		media	0.4-1.2s
Score glitch en burst >5%		baja	120ms
Byte-stream en burst >20%		alta	800ms
Search overlay placeholder		baja	cíclico
Connection lost retry		media	3s
Fase resurrection (rose→teal)		alta	1000ms

Anti-patrones — **dónde NO usar reactbytes**: (1) hover states — rompe el feel de "instrumento serio"; (2) charts/sparklines — compite con la data; (3) panel headers, ticker, left list rows — son scanneables, no narrativos; (4) cada keystroke en inputs — performance killer; (5) más de 2 instancias simultáneas — mata el efecto (cognitivamente saturante); (6) tooltips — deben aparecer instantáneamente; (7) empty states — mejor ASCII art estático que typewriter lento.

10.7
 ·
 Charts y visualizaciones

Chart	Dónde	Librería	Notas
Sparkline (60×16px)	fila de narrative list	custom SVG	Sin librería, path stroke 1.5px, área fill alpha 0.1
Velocity area chart	detail timeline	visx @visx/shape	Brush para zoom, marcadores de burst (CUSUM) y dips
Lifecycle track	detail top	custom SVG	4 nodos (◊●○○) con línea conectora, marca temporal animada
Heatmap (24h × 7d)	detail activity by time	custom SVG grid	Celdas 12×12px, color ramp slate→teal→amber según densidad
Network graph	graph view (g)	react-force-graph-2d	Nodos=accounts, edges=retweets. Cluster coloreado
Phase transition matrix	compare view	custom SVG	Tracks horizontales con 4 nodos por narrative
Overlaid velocity curves	compare view	visx	4 series con stroke dashed/solid/dotted
Acceleration strip	bottom bar (80px)	custom SVG	10 sparklines en fila, una por top-10 narrative

Por qué visx y no Recharts/Nivo/D3 puro: visx (Airbnb) es componible, primitives low-level, no asume layout → encaja con filosofía multi-panel custom. Tree-shakeable. React-first. Recharts asume layout de chart standalone (con padding, legend) — no encaja en paneles densos de 320px. Nivo bundle grande, estética default muy "SaaS". D3 puro demasiado imperative, sucio en React 19 con effects.

10.8
 ·
 Cuatro flujos de usuario principales

Flujo 1 — **Descubrir narrativa emergente** (objetivo: encontrar una narrativa antes del burst):

FLOW

```

[boot 3s] → [terminal default]
  ↓ user mira left panel sorted by σ(burst)
[ve ◊ FORMING con σ saltando 0.4→0.9]
  ↓ j/k para navegar, hover muestra mini-sparkline
[enter] → [detail expandido en center]
  ↓ ve timeline mostrando inflection point en últimos 30min
[confirmado early signal]
  ↓ s (star/subscribe)
[recibe alerta futura cuando phase → rising]
  ↓ g (graph view)
[ve network graph, identifica amplificadores clave]
  ↓ opcional: c (compare con narrativa similar pasada)
[ve overlaid curves, decide si actuar]

```

Flujo 2 — Monitorear narrativa activa (tracking en vivo de narrativa starred):

FLOW

```

[user tiene 3 narrativas starred, visibles en ticker]
  ↓ tab para ciclar entre starred en center panel
[ve detail de narrativa activa, stream derecho fluye]
  ↓ score cambia >5%
[glitch en el número, byte-stream si >20%]
  ↓ user hace click en burst moment en timeline
[chart rewind a ese instante, muestra tweets del burst]
  ↓ hover sobre tweet → tooltip con full context
[opcional: click → abre original en nueva tab]
```

Flujo 3 — Explorar narrativa decaída (entender por qué decayó, detectar resurrection):

FLOW

```

[f para abrir filter overlay]
  ↓ selecciona phase=decaying, sort by peak_time desc
[lista reordena, muestra narrativas decayed recientes]
  ↓ enter en una
[ve detail con lifecycle track completo + decay projection]
  ↓ r (resurrection probability)
[badge "RESURRECTION 12% (σ re-emerging 0.3)"]
  ↓ si probabilidad >20%, banner sutil "watch this"
```

Flujo 4 — Comparar narrativas (benchmarking entre 2-4 narrativas):

FLOW

```

[desde cualquier narrative, presiona c]
[narrative añadida a compare tray (max 4)]
  ↓ repite con 2-3 más
[enter sobre el tray]
  ↓ [vista 3: comparativa full screen]
[ve tabla lado a lado + overlaid curves + phase matrix + cross-correlation]
  ↓ descubre p alta con lag → "ETH follows TAYLOR by 12min"
[insight accionable: usar TAYLOR como leading indicator para ETH]
```

10.9 · Keyboard-first — shortcuts vim

Key	Acción	Key	Acción
j / ↓	siguiente narrative	s	star/subscribe
k / ↑	anterior narrative	g	graph view
enter	abrir detail	c	add to compare
esc	cerrar overlay / detail	f	filter overlay
/	abrir search (cmdk)	r	resurrection probability (en decaying)
tab	ciclar starred narratives	m	toggle sound

Key	Acción	Key	Acción
Shift+Tab	reverse cycle	?	help overlay

10.10 · Anti-patrones (lo que NO haremos)

- ▶ **No glassmorphism ni blur.** La terminal es opaca. El blur consume GPU, resta contraste y comunica "SaaS genérico". Bordes de 1px sólidos dan toda la jerarquía.
- ▶ **No card-based dashboard genérico.** Prohibido el grid de cards flotantes con shadow. Los paneles comparten bordes (1px #21262D), no hay padding alrededor del layout. Es Bloomberg, no Notion.
- ▶ **No gradientes decorativos.** El único gradiente permitido es el área fill de un chart (alpha 0.1 del accent). Prohibido gradientes en backgrounds, botones, badges.
- ▶ **No animaciones de carga largas (>1s).** Skeleton inmediato desde el primer frame. El boot typewriter es la única excepción (3-5s) y es opcional (skip con cualquier tecla).
- ▶ **No modales centrados intrusivos.** Los overlays son laterales (slide from right) o inline expansion. Único modal centrado: search cmdk.
- ▶ **No dark/light toggle.** La terminal es dark-only por definición conceptual. Agregar un toggle rompe la metáfora.
- ▶ **No emoji como iconos.** Los iconos son símbolos mono: ▲ ▽ ◇ ● ◆ ■. Unicode puro, render consistente.

11 Roadmap, Métricas y Riesgos

Roadmap en 3 fases: MVP 4 semanas (demostrar lead time), v1 3 meses (anti-gaming serio y multi-rol), v2 6 meses (intelligence platform). North Star metric: lead time accionable medio. Tabla de riesgos con mitigaciones. Compliance posture: hashing SHA-256 con salt rotatorio, opt-out, bot badge con disclaimer.

11.1 · Roadmap — 3 fases

Fase	Timeline	Objetivo	Features clave	No entra
MVP	Semanas 1-4	Demostrar lead time	Ingest 3 fuentes (Twitter+GDELT+Reddit), scoring simple (v,a,H), 4 fases, heurísticas anti-gaming capa 1 (8 reglas), dashboard web, 1 workflow (OSINT), alertas email+webhook	GNN, multi-rol, integraciones exchange, multi-idioma profundo
v1	Meses 2-3	Anti-gaming serio + multi-rol	Isolation Forest + GNN GraphSAGE, cross-validation bayesiana completa (5 fuentes), 5 dashboards multi-rol, origin quality completo, alertas multi-canal (Slack/Teams/Discord/SMS), API pública, pricing tiers	Multi-idioma profundo, predictive modeling
v2	Meses 4-6	Intelligence platform	Multi-idioma (ES/EN/PT/FR/DE/RU/ZH), Telegram/BlueSky/Mastodon ingestion, predictive modeling LSTM/Transformer, anomaly discovery sobre embedding space, graph exploration UI (Maltego-like), marketplace de watchlists, white-label para agencias	—

11.2 · Métricas North Star y KPIs por versión

La métrica rectora del producto es el **lead time accionable medio**: tiempo entre nuestra alerta de narrativa emergente y el burst público en Twitter Trending, ponderado por precision. Si sube, ganamos. Si baja, somos un clon caro de Twitter Trending.

Métrica	Definición	Target MVP	Target v1	Target v2
Lead time accionable medio	Tiempo entre detección forming y peak público, ponderado por precision	>1h en 50% casos	>3h en 70%	>6h en 80%
Precision@k (LEGIT)	De LEGIT, cuántas reales	>0.80	>0.90	>0.93
Recall (BOT_CAMPAIGN)	De bots removidos por Twitter, cuántos detectamos antes	n/a	>0.60	>0.75
False-positive bot rate	Cuentas legítimas marcadas como bot	<5%	<2%	<1%
NPS por rol	Net promoter score (periodista, trader, marca, OSINT, político)	—	>40	>55
Daily Active Watchlists	Watchlists activas/día	—	>500	>5000
Time-to-insight	Tiempo desde abrir dashboard hasta acción	<5 min	<2 min	<1 min
Conversion free → pro	% que pasa a pro	—	>5%	>10%

Métrica	Definición	Target MVP	Target v1	Target v2
Churn mensual	% pro que cancelan	—	<5%	<3%

11.3 · Tabla de riesgos y mitigaciones

#	Riesgo	Prob	Impacto	Mitigación
1	Ban masivo de Twitter por detección de patrón de scraping	Alta	Crítico	Rotación proxies residenciales, 10 sesiones con fingerprints distintos (CloakBrowser stealth=2), delays jitter 5-15s, max 20 req/min, circuit breaker 5 fails → 60s cooldown. Fallback a Nitter si Twitter cae
2	Cluster spam: campaña coordinada genera narrativa falsa de alto score	Media	Alto	trash_penalty con 5 sub-factores multiplicativos: ratio autores nuevos >70% en 1h → -0.3; bot_score >0.6 → -0.4; cosine >0.95 (copypasta) → -0.5. Validación humana opcional con flag_for_review cuando score >70 pero trash_penalty <0.6
3	Latencia de embeddings dispara watermark >35s	Media	Medio	Sidecar Python con modelo ONNX-quantizado, batch de 50, GPU opcional (T4). Cache LRU de embeddings por hash de contenido. Si p95 >3s → reducir ventana tier-2 a 200 mentions o activar modo GPU
4	Drift del modelo de clustering: embeddings pierden relevancia con nueva jerga	Baja	Medio	Re-entrenar/re-evaluar quarterly con dataset etiquetado (200 narrativas/mes muestreo humano). Métrica: ARI entre clusters viejos vs nuevos >0.7. Swap a bge-m3 si drift >30%
5	Límite de conexiones SSE en single Node process	Media	Medio	Default Node = 1K sockets. Con keepAliveTimeout + maxConnections tuneado llegamos a 2-3K. Si >5K usuarios concurrentes → sticky sessions + N instancias detrás de LB (Redis Streams fan-out)
6	Postgres bloat por alta inserción en mentions	Media	Alto	Particionado mensual + DROP PARTITION para retención 90d. Autovacuum agresivo (scale_factor=0.05). BRIN en vez de B-tree para timestamps. mentions no se UPDATEA. pg_repack mensual
7	GDELT lag: actualiza cada 15min, a veces tarda 2h	Alta	Bajo	No bloquear pipeline por GDELT. Marcar menciones GDELT con freshness_penalty. UI muestra "delayed source" badge para GDELT vs Twitter/Reddit real-time
8	CloakBrowser 0.5.2 desactualizado o cambios en DOM de Twitter	Media	Alto	Pin en Dockerfile, tests E2E nightly que validan selector article[data-testid="tweet"]. Circuit breaker detecta 0 resultados en N requests → alerta PagerDuty → fallback a API no-oficial o Nitter
9	Reddit OAuth2 cambia pricing (estilo 2023)	Media	Alto	Abstracción lista para swap a pushshift.io alternatives (rar.md, arctic-shift), RSS como último recurso
10	GDPR / privacy concerns por author data	Media	Alto	Hashing SHA-256 con salt rotatorio mensual (no reversible, no PII bajo GDPR Art.4). Author handle solo 30d en Redis. Opt-out público. Documento compliance posture

11.4 · Ética y legalidad

El scraping de Twitter está prohibido por los ToS de X Corp. La Terminal de Viralidad lo hace igual porque: (a) no redistribuye contenido, solo métricas y links; (b) no compite con la API oficial (caso de uso distinto); (c) el scraping de datos públicos para análisis ha sido sostenido en hiQ Labs vs LinkedIn (US 9th Circuit 2022). Sin embargo, el riesgo legal existe y se documenta en el "compliance posture" interno.

GDPR: los datos de cuentas de usuario son personales en UE. Mitigaciones: hashing SHA-256 con salt rotatorio mensual (no reversible, no es PII bajo Art. 4); author handle display solo 30 días en Redis (no en Postgres); opt-out público via página dedicada; reputation se computa agregado, no se guarda per-author PII. Bot badge con disclaimer explícito: "modelo probabilístico, no afirmación de hecho" — para evitar demandas por difamación en false-positives.

TOS restrictivos contra surveillance estatal no-democrática. Auditoría de sesgos trimestral por idioma y tipo de cuenta (el modelo anti-gaming puede tener falsos positivos contra cuentas minoritarias o no-inglés). Rate-limit + human-in-the-loop: ninguna acción automática contra cuentas. Transparencia metodológica: paper técnico público. DPA/DPIA GDPR + DPO si >UE users.

11.5 • Modelo de negocio

Tier	Precio	Features	Target
Free	\$0	5 alertas/día, 1 watchlist, dashboard básico, solo Twitter+GDELT	Lead magnet, capturar PMF
Pro	\$49-199/mes	Multi-rol (5 dashboards), alertas ilimitadas, 5 fuentes, Slack/Teams webhook, API read-only	Periodistas, traders indie, OSINT freelance
Team	\$499-999/mes	5 seats pro, watchlists compartidas, API read-write, SSO, export CSV/JSON	Boutiques de inteligencia, newsrooms pequeños
Enterprise	\$1000+/mes	White-label, SLA 99.9%, on-prem option, custom sources, dedicated support, DPA	Agencias, gobiernos democráticos, enterprises

El precio se justifica no por la cantidad de datos sino por el lead time accionable: si un trader promedio genera \$500 por operación con 2h de anticipación, cualquier precio menor a eso es obvio. Si un periodista publica una exclusiva 24h antes que la competencia, el valor generado supera cualquier suscripción mensual. Si una marca detecta una crisis 6h antes y reduce MTTR 50%, el ahorro supera \$10K. La pricing está alineada con valor entregado, no con costo de provisión.

11.6 • Cierre

NORTE ESTRATÉGICO FINAL

*Sin la combinación de **lead time** + **legitimacy badge** + **cross-source attribution**, el producto es un clon caro de Twitter Trending. Con esos tres diferenciadores juntos, es una herramienta de intelligence con lead time accionable para cinco ICPs distintos. El algoritmo desdoblado $Vel \times Mat \times Pen \times Decay$ con HMM de 4 fases y CUSUM+Kleinberg para bursts, combinado con cross-validation bayesiana entre 5 fuentes y penalty multiplicativo de 5 sub-factores, es la receta técnica que sostiene esa promesa de valor. El MVP en 4 semanas demuestra el lead time; v1 en 3 meses cierra anti-gaming y multi-rol; v2 en 6 meses es intelligence platform.*

La terminal se construye ahora. Los cinco agentes especialistas alinearon arquitectura, algoritmo, UX/UI, fuentes y estrategia. El siguiente paso es el MVP: 4 semanas, 3 fuentes, 1 ICP (OSINT), demostrar que detectamos narrativas emergentes 1+ hora antes que Twitter Trending. Si lo logramos, el resto del roadmap se ejecuta con data real, no con hipótesis.